

Universiteit Leiden

Master Computer Science

Subgoal-Chain-Driven Long-Horizon Decision Making in LLM Agents with Autonomous Reasoning and Programmatic Verification

Name:	Yijie Wang
Student ID:	s4454502
Date:	25/08/2026
Specialisation:	Data Science
1st supervisor:	Mike Preuss
2nd supervisor:	Matthias Müller-Brockhausen

Master's Thesis in Computer Science

Leiden Institute of Advanced Computer Science (LIACS)
Leiden University
2333 CA Leiden
The Netherlands

Abstract

Long-horizon decision making by large language model (LLM) agents becomes especially difficult in tasks with irreversible resource constraints, where locally reasonable actions may still be globally harmful. To study this problem, a simplified Mota-style dungeon environment is constructed in which keys, doors, floor transitions, and final objectives create long-term dependencies. A series of LLM-based agents is then designed and compared, progressing from local action and route scoring to subgoal-chain reasoning, programmatic reachability verification, and reflection. Early approaches frequently suffered from goal drift and poor resource awareness. To address these limitations, LLM-generated subgoals are combined with BFS-based reachability verification, allowing intermediate decisions to be checked against the final objective before execution. An advanced version also includes reset and reflection mechanisms that record failure experiences as reusable lessons. The results show that explicit subgoal chains and programmatic verification improve decision consistency and reliability in long-horizon, resource-constrained tasks. More importantly, the study proposes and analyses a long-horizon decision-making framework that combines autonomous LLM-generated subgoal reasoning with deterministic, BFS-based reachability verification. Through an evolutionary sequence of agent versions, it reveals the role of subgoal chains in mitigating goal drift, preserving the salience of intermediate objectives, and improving success in resource-constrained tasks. Overall, the study provides experimental evidence and methodological guidance for building more reliable long-horizon LLM agents.

Contents

Chapter 1 Introduction **5**

Chapter 2 Background and Related Work **7**
2.1 Large Language Model Agents 7
2.2 LLM Agents in Game Environments 7
2.3 Long-Horizon Decision Making and Goal Drift 7
2.4 Subgoal Chains 7
2.5 External Verification for LLM Planning 8
2.6 Structured Outputs 8

Chapter 3 Experimental Environment and Problem Setting **8**
3.1 The Mota Puzzle Environment 8
3.2 Environment Simplification and Design Rationale 9
3.3 Formal Environment Definition and State Representation 10
3.4 Problem Statement: Subgoal Dependency Structure 12

Chapter 4 Methodology **13**
4.1 Decision-Making Loop in Agent Framework 13
4.2 From Scoring to Subgoal Chains 14
4.3 Generation I: Action Scoring 15
4.4 Generation II: Program-Constructed Subgoal Chains 16
4.5 Generation III: LLM-Constructed Chains with Programmatic Verification 17
4.6 Mechanisms 18
4.7 Summary of Validation and Recovery Mechanisms 19
4.8 Summary of Architectural Evolution 19
4.9 Representative Implementation Versions 20

Chapter 5 Implementation **21**
5.1 System Architecture and Module Separation 21
5.2 Prompt Design and LLM Interface 22
5.3 Programmatic Verification: Reachability Checking 23
5.4 Human Supervision 23
5.5 Deadlock Detection and Reflection Memory 24
5.6 Logging and Experiment Tracing 25

Chapter 6 Experimental Setup **25**
6.1 Experimental Goals 25
6.2 Task Configurations: Floor Count as a Horizon-Length Variable 26
6.3 Agent Versions Evaluated 27
6.4 Models Evaluated 27
6.5 Failure Reasons 27

Chapter 7 Results and Discussion **28**
7.1 Result for RQ1 28
7.2 Result for RQ2 37
7.3 Cross-Model Comparison on the Five-Floor Task 41

7.4 Cross-Model Comparison on the Seven-Floor Task 43
7.5 Why did the weaker 20B model fail on the 5-floor task, yet achieve one success on the 7-floor task under Agent70b1? 44

Chapter 8 Conclusion **46**
8.1 Future work 47

Appendices **51**

Chapter A Mota1 Agent13 and Agent14 System Prompt **51**
A.1 `mota1_agent13` System Prompt 51
A.2 `mota1_agent14` System Prompt 51

Chapter B Full Logs and Traces: `mota1_agent13` vs. `mota1_agent14` **52**
B.1 `mota1_agent13` Run Log 52
B.2 `mota1_agent14` Run Log 55
B.3 Decision-Trace Excerpts at the Critical Fork (Step 6) 57

Chapter C Prompt Templates **58**
C.1 Planning Prompt 58
C.2 Reflection Prompt 62
C.3 Human-Interaction Prompt 62

1 Introduction

LLM agents are increasingly used in tasks that require sequential decision-making, such as web navigation [1], robotics [2], and tool use [1] [2], as well as game playing and game-based embodied environments [3, 4, 5]. They are no longer expected only to answer questions, but also to observe environments, formulate goals, execute actions, and adjust their plans based on feedback. However, these tasks often require long-horizon reasoning: the agent must not only decide what is useful now, but also preserve progress toward a final objective over many future steps.

Long-horizon decision making remains difficult for LLM-based agent frameworks. A model may generate a locally plausible explanation for an action while failing to consider its future consequences [6]. This problem becomes especially important in environments with irreversible resource constraints. In such settings, an action may consume a key resource, open one path while blocking another, or make a later objective unreachable. Therefore, a locally valid action can still be globally harmful. The present study focuses on such failures, especially goal drift [7] [8] and resource-planning errors. Goal drift occurs when the decision-making agent gradually shifts from the final objective to locally attractive intermediate goals, such as opening a nearby door or collecting a nearby item, without verifying whether the action still supports the final task.

To study this problem in a controlled setting, we use a simplified Mota dungeon puzzle environment inspired by Tower of the Sorcerer [9]. The environment introduces irreversible key-door dependencies across multiple floors, requiring the agent to reason not only about immediate reachability but also about whether limited resources should be preserved for future objectives. This makes long-horizon planning failures easy to observe and diagnose, since a locally valid action can render the final objective unreachable.

The long-horizon decision-making and reasoning capabilities of LLM-based agent frameworks are investigated here through an evolutionary sequence of designs. Early versions rely on natural-language interaction, action scoring, and route scoring, where the model evaluates candidate actions or paths suggested by the program. These experiments did not reliably solve the task, but they helped reveal important failure modes: the model often focused on local progress, overlooked cross-floor resource dependencies, or failed to maintain feedback over later decisions. Later versions move toward target-level planning, structured prompting, subgoal-chain reasoning inspired by chain-of-thought prompting [10] and prior game-AI research in StarCraft [11], and experience reflection inspired by verbal reflection memory and broader agent-memory research [12, 2]. Throughout this development process, programmatic search and LLM-based decision making were combined in different ways. Prior work has shown the limitations of LLM self-critique and the benefits of sound external verification [13, 14]. In the Generation III architecture developed here, the language model first proposes a target, and breadth-first search (BFS) [15] is then used to verify whether that target is reachable before execution.

Early versions of the agent relied mainly on scoring mechanisms: the program generated candidate actions or routes, and the language model selected among them using natural-language reasoning. These experiments showed that LLM-based interaction could sometimes improve local decisions and help the model explain how a final objective could be divided into smaller intermediate goals. However, these intermediate goals were not maintained consistently during execution. As shown by the interaction logs analysed in Chapter 7, once the agent returned from discussion to action selection, the model some-

times abandoned its previously stated intermediate goals and selected locally attractive but globally harmful actions.
This recurring pattern motivated a transition from purely local scoring toward architectures that maintain an explicit subgoal-chain representation. Although Generation II continued to use route scoring, the candidate routes were organised according to a dependency structure. Generation III went further by asking the language model to construct the subgoal chain and propose the next target. The general motivation for representing intermediate reasoning steps is related to chain-of-thought prompting and prior work on subgoal-based long-horizon agents [10, 16]. However, the subgoal chain used here represents task-level objectives and dependencies rather than an unrestricted natural-language reasoning trace.
In the Mota environment, a subgoal chain connects the final objective with the required resources, doors, floor transitions, and intermediate targets. Its purpose is to keep these dependencies visible during execution, reduce goal drift, and make individual decisions easier to interpret as parts of a longer-term plan. The advanced agent also explores whether failures recorded in historical logs can be converted into reusable textual lessons, following the general idea of verbal reflection memory [12]. These lessons are included in later planning prompts after a deadlock-triggered reset.

The study addresses the following research questions:

- RQ1: What kinds of long-horizon decision failures occur in LLM agents under irreversible resource constraints?
- RQ2:How does prompt design influence the decision-making behavior of LLM agents in long-horizon tasks?
- RQ3: Does explicit subgoal-chain reasoning reduce goal drift and resource-planning errors?
- RQ4: Can programmatic verification and experience reflection improve the reliability of LLM agents in resource-constrained environments?

The remainder of this study is structured as follows. Chapter 2 reviews related work on LLM agents, long-horizon planning, goal drift, subgoal decomposition, and structured output. Chapter 3 defines the simplified Mota environment and the task setting. Chapter 4 presents the agent methodology and the evolution of decision representations. Chapter 5 describes the implementation. Chapter 6 explains the experimental setup. Chapter 7 reports the results, discusses the findings, and limitations. Chapter 8 concludes the study and discusses directions for future work.

2 Background and Related Work

2.1 Large Language Model Agents

In an LLM agent setting, the model observes an environment state, reasons about possible actions, selects an action or goal, and receives feedback from the environment [2]. ReAct [17] introduced an agent framework that interleaves reasoning and action, allowing the model to use new observations when making subsequent decisions. This observe–reason–act loop provides the basic interaction pattern used by many LLM agents, including the Mota agents developed in this study. Reflexion [12] extended this idea by converting feedback from unsuccessful attempts into textual reflections that can guide later attempts. Inspired by this memory-based approach, the reflection-enabled Mota agent converts deadlock-triggering failures into short textual lessons.

2.2 LLM Agents in Game Environments

To study intelligent agents and large language models, many previous works [3, 4, 5, 18] have used games and interactive environments as experimental testbeds. These environments provide clear states, actions, rules, and objectives, making it possible to observe how models make decisions over multiple steps. For example, ScienceWorld [18] provides an interactive text-based scientific environment for evaluating whether agents can apply scientific knowledge through reasoning in unfamiliar situations, rather than relying primarily on patterns encountered during prior training. DEPS [5], on the other hand, studies long-horizon planning and interactive decision-making in the Minecraft environment, where an agent must continuously interact with the world and adjust its plan according to the current state. Compared with these environments, the simplified Mota environment used in our experiment focuses specifically on irreversible key-door dependencies, where a locally valid action can consume a resource required for a later objective. This makes it particularly suitable for studying long-horizon reasoning, resource planning, and whether an agent can maintain consistent decisions when earlier actions have delayed consequences.

2.3 Long-Horizon Decision Making and Goal Drift

Our experiments independently reveal a similar pattern: intermediate actions can become misaligned with the final objective over long decision horizons. This observation is consistent with recent work on myopic long-horizon planning[6, 16] and goal drift[7, 8] in LLM agents. Prior work on goal drift shows that language-model agents may gradually deviate from their original task when exposed to long interaction histories, competing local goals, or ambiguous feedback. This issue is especially problematic when actions have irreversible consequences. In the Mota setting, opening a nearby door may appear to be progress, but it can cost a key that is required by a later critical door. This makes goal drift observable as a concrete resource-planning failure.

2.4 Subgoal Chains

The proposed subgoal-chain representation is related to two lines of prior work. First, research in game AI commonly decomposes complex game-playing problems into smaller strategic, tactical, and operational components [11], providing a broader motivation for

representing intermediate objectives explicitly. Second, chain-of-thought prompting [10] demonstrates the usefulness of organizing intermediate reasoning steps into a sequence leading toward a final result. Drawing on these ideas, the present study develops a task-specific subgoal-chain representation that links the final objective to the required resources, doors, floor transitions, and immediate targets. Unlike conventional chain-of-thought reasoning, however, the elements of the chain are explicit task-level objectives and dependencies rather than unrestricted natural-language reasoning traces. Related work has also applied subgoal decomposition to long-horizon LLM agents [16]. That work combines online planning through subgoal decomposition with milestone-based rewards for reinforcement learning. Although its environment and training method differ from those used here, its results provide further evidence that explicit subgoals can support long-horizon agent behaviour, which is consistent with the motivation behind the subgoal-chain design proposed here.

2.5 External Verification for LLM Planning

Language models do not always produce valid or executable plans [19]. Stechly et al. [14] found that sound external verification is more reliable than asking the language model to verify its own output.

Generation III uses a similar separation between language-model planning and programmatic checking. The language model proposes a subgoal chain and a primary target, after which BFS [15] checks whether the target is reachable from the current game state. This check verifies only target reachability, not the correctness of the complete subgoal chain.

2.6 Structured Outputs

Structured outputs are important in LLM-based agent systems because formats such as JSON provide a machine-readable interface between the language model and the surrounding agent framework. Previous work has shown that structured-output generation is useful for software integration, but that LLMs do not always reliably follow the requested format [20, 21]. In particular, benchmark studies have found substantial variation in format adherence across models, tasks, and prompting strategies [21, 22].

Constrained decoding can improve output reliability by restricting generation to a predefined structure, such as a JSON Schema. Recent work has systematically evaluated these methods for structured-output generation [23]. This is relevant here because the agent relies on structured model responses for programmatic decision making.

In our framework, the LLM is instructed to return decisions in JSON format, with fields such as the selected target, proposed subgoal chain, and other decision-related information.

3 Experimental Environment and Problem Setting

3.1 The Mota Puzzle Environment

The experimental environment used in this study is inspired by "Magic Tower", a genre of turn-based, fixed-numeric-value strategy RPG that originated in Japan. The genre traces back to Tower of the Sorcerer [9], prototyped in 1994 on MS-DOS and officially released

in 1996 on the PC-98 platform by Caravan Software. The original game established the genre's core gameplay loop: the player controls a hero who must calculate numeric attack and defence values to determine which monsters can be safely defeated, and must plan a route through the tower that collects keys and clears monsters in an order that ultimately allows the rescue of a trapped companion or princess. This foundational design where floor traversal is gated by a dependency web of monsters (requiring sufficient combat stats) and colour-coded keys and doors established the puzzle structure that the genre is known for: progress requires reasoning about long chains of prerequisites rather than simple point-to-point navigation. This dependency structure makes the genre a natural, low-cost testbed for studying long-horizon subgoal reasoning in artificial agents: unlike open-ended sandbox environments (e.g., Minecraft) [3], a Magic Tower map has a small, fully specified rule set and a verifiable ground-truth solution, which makes it possible to precisely diagnose why an agent fails a particular subgoal.



Figure 1: Example floor of the original Mota puzzle environment. Walls, colour-coded keys and matching doors, and staircases connecting floors are shown.

3.2 Environment Simplification and Design Rationale

As described in Section 3.1, traditional Magic Tower games usually combine two related systems. The first is a combat system, where the player uses attack and defence values to decide which monsters can be defeated safely. The second is a key-and-route puzzle, where the player must plan the order of collecting coloured keys, opening matching doors, and using staircases between floors. These systems are not independent in the original game. The result of a fight can affect whether a route is still possible, and the order in which monsters are defeated can also affect resource planning.

This study removes the combat system completely. The environment used in the rest of this work keeps only the key, door, and staircase dependencies. It has no monsters, health points, attack values, or defence values. The agent can only perform four basic movement actions. The only irreversible resource event is opening a door, which permanently uses one matching key.

This simplification is an intentional design choice. Combat would add two extra factors that would make the analysis of long-horizon planning more difficult. First, it would add a numerical optimisation problem, such as balancing attack and defence values. This is separate from the dependency reasoning studied in this work. Second, many versions of the game use random combat outcomes. This would make it difficult to decide whether a failed run was caused by a planning error or by an unlucky combat result.
By focusing only on key, door, and staircase dependencies, the study can link each failure in the later experiments to a planning or verification problem, rather than to an uncontrolled combat result.

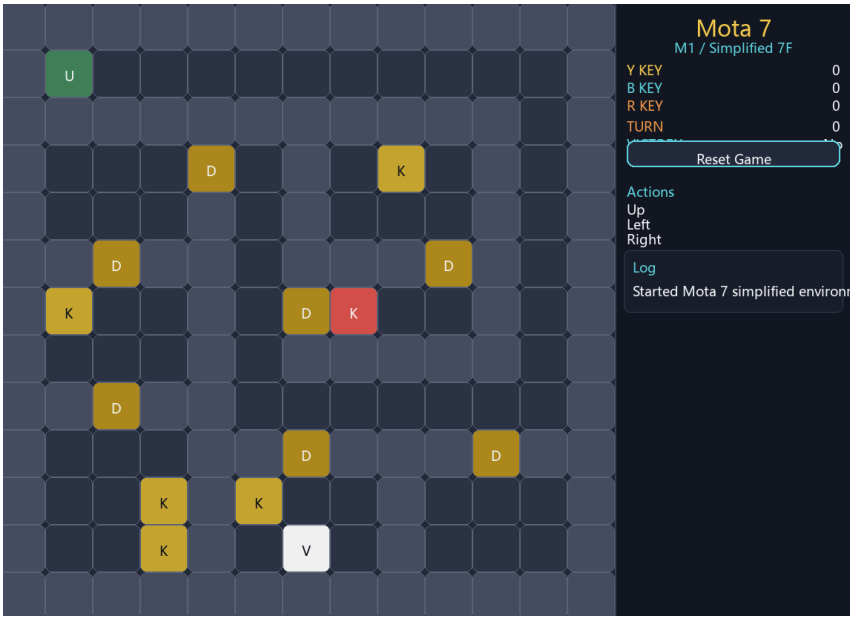


Figure 2: Example floor of the simple Mota puzzle environment. Walls, colour-coded keys and matching doors, and staircases connecting floors are shown.

3.3 Formal Environment Definition and State Representation

Map structure. The environment has seven floors, named M1 (floor 1) to M7 (floor 7). Each floor is a 13×13 grid of tiles. The floors are linked by staircases. When the agent reaches a staircase tile, it moves to the matching staircase tile on the floor above or below. The final goal is to reach a fixed goal tile on floor M7 As a concrete example, JSON information shows the tile layout of floor M1 using the symbols defined below.

Markov Decision Process Formulation. The simplified Mota environment can be formalised as a deterministic, fully observable Markov Decision Process [24]. It is important to note, however, that the agent studied in the proposed approach is not a reinforcement-learning policy obtained through reward-based training; rather, it is a hybrid decision framework in which high-level decisions are produced by an LLM and validated through programmatic verification. The MDP formalism introduced below therefore serves as a precise vocabulary for describing states, transitions, and the goal condition, rather than as the basis for a learned value function or policy-gradient method. Formally,

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, s_0, \mathcal{G}),$$

```
#############
#U.........#
###########.#
#...D.#.K.#.#
#...#.#...#.#
##D##.###D#.#
#K..#.Dr..#.#
#...#.#####.#
##D##.......#
#...##D###D##
#..K#K..#...#
#..K#...#...#
#############
```

Symbol	Meaning
`#`	Wall (impassable)
`.`	Empty, walkable tile
`K` / `b` / `r`	Yellow / Blue / Red key
`D` / `B` / `R`	Yellow / Blue / Red door (requires and consumes one matching key)
`U` / `N`	Staircase up / down (moves the agent between floors)

Table 1: Tile symbols used in the simplified Mota environment (right), illustrated with the layout of floor M1 (left).

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $T : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$ is a deterministic transition function, $s_0 \in \mathcal{S}$ is the initial state, and $\mathcal{G} \subset \mathcal{S}$ is the set of goal states.
A state $s \in \mathcal{S}$ is a tuple

$$s = (p, f, k, D),$$

where $p = (x, y) \in \{1, \ldots, 13\}^2$ is the agent's position on the current floor, $f \in \{1, \ldots, 7\}$ is the current floor index, $k = (k_Y, k_B, k_R) \in N^3$ is the number of yellow, blue, and red keys currently held, and $D \subseteq \mathcal{D}$ is the subset of doors (from the fixed set of all doors $\mathcal{D}$ on the map) that have already been opened. Note that D is part of the *environment* state rather than the agent's private state: once a door is opened, it remains open for the rest of the episode, which is precisely the source of the irreversibility discussed in Section 3.3.
The action space is $\mathcal{A} = \{\text{up}, \text{down}, \text{left}, \text{right}\}$, with $|\mathcal{A}| = 4$. The transition function T is defined by the tile rules given below: moving into a wall leaves s unchanged; moving into an empty tile updates p; moving onto a key tile updates p and increments the corresponding component of k; moving onto a door tile d with a matching key updates p, decrements the corresponding component of k by one, and sets $D \leftarrow D \cup \{d\}$; moving onto a staircase tile updates f and relocates p to the linked tile on the adjacent floor. Because T contains no stochastic component and the full state s is observable at every turn, $\mathcal{M}$ is deterministic and fully observable: the only source of difficulty is combinatorial rather than perceptual or probabilistic.
The initial state is $s_0 = (p_{\text{start}}, 1, (0, 0, 0), \emptyset)$, and the goal set is

$$\mathcal{G} = \{\, s = (p, f, k, D) \in \mathcal{S} \mid p = p^*,\ f = 7 \,\},$$

i.e., any state in which the agent occupies the designated goal tile p^* on floor M7, independent of k and D. An episode is a sequence of states $s_0, s_1, \ldots, s_t$ generated by $s_i = T(s_{i-1}, a_i)$ for a chosen action sequence $a_1, \ldots, a_t \in \mathcal{A}$, terminating successfully if $s_t \in \mathcal{G}$ for some t within the turn budget, and terminating unsuccessfully otherwise.

Action space. The agent selects one of four deterministic movement actions at each turn: *up*, *down*, *left*, *right*. Moving onto a door tile without the matching key, or onto a

wall tile, is not permitted; moving onto a key tile picks up the key automatically; moving onto a door tile with a matching key consumes one key and opens the door; moving onto a staircase tile transitions the agent to the corresponding tile on the adjacent floor.

Win condition. An episode terminates successfully when the agent reaches the designated goal tile on floor M7(M3,M5). An episode may also terminate unsuccessfully if the agent exceeds a fixed turn budget or a fixed number of failed re-planning attempts (Chapter 4).

State variables. The environment state at any point in time is fully described by: the agent's current (x, y) coordinates; the current floor; the number of yellow, blue, and red keys currently held; and the elapsed turn count. This state is sufficient to determine which actions are currently valid, since the underlying map layout is static and known in advance.

Irreversible Resource Constraints. A defining property of this environment is that keys are limited and irreversible resources: once a key is used to open a door, it cannot be recovered. As a result, an action that appears locally useful, such as opening a nearby door, may consume a key required by a later dependency and thereby make the final goal unreachable. Because the environment provides no immediate failure signal when this happens, the consequences of such mistakes may only become visible many turns later. This makes the task well suited to studying long-horizon decision making, goal drift, and resource-planning errors.

Agent-facing representation. The raw grid state is not passed to the language model directly as a matrix. Instead, the agent converts the complete current game state into a structured textual representation. For Generation III, this representation includes the player's current position and floor, the current key inventory, the current maps of all floors, and the remaining keys, doors, and staircases relevant to planning. This information is included in the prompt together with a description of the movement and key–door rules.

Modular architecture. The environment simulator (*engine*), the decision-making component (*agent*), and the visualisation/interaction layer (*interface*) are implemented as separate modules that communicate through the shared state representation described above. This separation is introduced here for context and is described in full in Chapter 5 (Implementation).

3.4 Problem Statement: Subgoal Dependency Structure

Formal definition. Let a *subgoal* g be a condition on the environment state defined in Section 3.3 for example, "hold at least one yellow key," "the yellow door on floor 2 is open," or "the agent is standing on the down-staircase of floor 3." Each subgoal g_i has a (possibly empty) set of preconditions $\text{pre}(g_i)$, which are themselves subgoals that must already be satisfied before g_i can be achieved. For instance, opening a door requires the precondition of holding a matching key, which in turn requires the precondition of having reached the tile on which that key is located.

Completing the environment corresponds to reaching a single terminal subgoal g^*: the agent is standing on the goal tile of floor M7. A *subgoal chain* is an ordered sequence

$$g_1 \to g_2 \to \cdots \to g_n = g^*$$

such that, for every i, all preconditions in $\mathrm{pre}(g_i)$ are satisfied by some g_j with $j < i$. Executing this chain requires the agent to translate each subgoal into a sequence of primitive movement actions (Section 3.3) that reach the corresponding tile, and, where applicable, to consume the resource associated with that subgoal (e.g., a key).

Why an incorrect chain is catastrophic. Because key consumption is irreversible, a subgoal chain that reaches a door subgoal g_i before satisfying all of its true preconditions or that consumes a key on a door not required by g^* does not simply fail locally. It can permanently remove a precondition required by some later subgoal g_k, $k > i$, making g^* unreachable for the remainder of the episode even though the agent can still move freely between the remaining accessible tiles. This is what Chapter 1 refers to as a resource-planning failure caused by goal drift: the agent commits to a subgoal that is locally attractive (e.g., the nearest reachable door) without verifying its position in the correct dependency order relative to g^*.

4 Methodology

4.1 Decision-Making Loop in Agent Framework

How, then, does the LLM actually play the game through the agent? Rather than interacting with the game engine directly, the LLM operates through an intermediate software layer referred to here as the agent. In each decision cycle, the game engine first exposes the current game situation as a structured game state, including relevant information such as the player position, available resources, and map configuration. The agent then converts this state into a model request by embedding the structured information into a prompt together with task instructions. Based on this input, the LLM returns a structured decision describing the next intended action or subgoal. Finally, the agent interprets this output and translates it into executable in-game actions, which are sent back to the game engine. In this way, the LLM does not act on the environment directly, but plays the game indirectly through the agent, which serves as the interface between language-based reasoning and game execution. This design was motivated in part by prior work showing that large language models can be studied as game-playing agents in complex game environments such as *Slay the Spire*, which helped establish the feasibility of using language-driven decision making for gameplay [4].

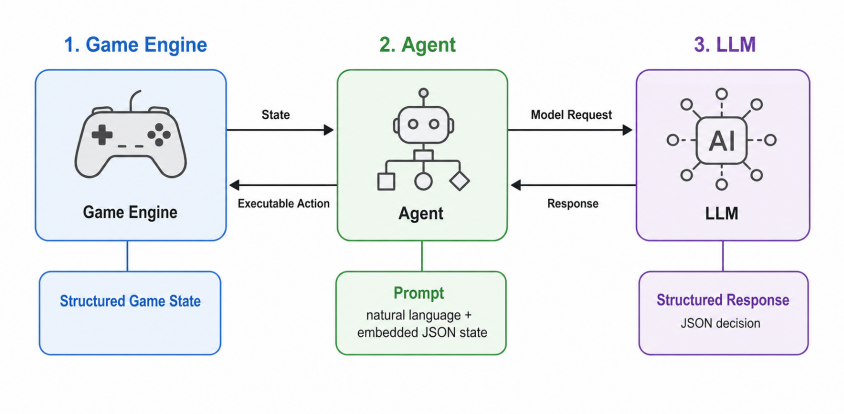


Figure 3: Communication loop between the game engine, the agent, and the LLM.

4.2 From Scoring to Subgoal Chains

The transition from purely local action scoring toward architectures with explicit subgoal chains was motivated by a recurring failure mode observed during development: goal drift in long-horizon decision making. The experimental logs repeatedly showed that the LLM could move away from an earlier objective and select actions that appeared locally useful but conflicted with longer-term requirements. Related problems with maintaining goal alignment across multi-turn interactions have also been studied in LLM-based user simulators for conversational AI [7]. Although that setting differs from game playing, it illustrates the broader difficulty of maintaining a stable objective over an extended interaction.

In Generation I, the LLM did not maintain an explicit dependency structure connecting the current action to the final objective. Instead, it scored all currently legal movement actions, using up to five locally available candidate targets supplied by the program as additional guidance. This made the decision process highly reactive. The agent retained a short history of recent actions, selected targets, and execution results, but the model's reasoning was not preserved as a structured commitment or planning mechanism for later decisions.

Human supervision experiments provided further evidence of this problem through natural-language discussion between the human operator and the LLM via the agent interface. These interactions showed that the model could often describe useful intermediate objectives when prompted. For example, it could explain that a key should be preserved for a later door or that a key on a lower floor had to be collected before progress could continue on a higher floor. However, these verbally stated objectives were not reliably maintained after the agent returned to action selection. After several additional decisions, the model sometimes returned to actions that appeared immediately attractive but did not support the previously identified dependency.

These observations suggested that the difficulty was not only whether the model could identify subgoals, but also whether those subgoals remained visible and influential throughout multi-step execution. The logs suggest that a subgoal stated once during an interaction could later be displaced by nearby reachable targets or by repeated attention to the general final objective. Consequently, apparent local progress could take priority over the dependencies required for eventual task completion.

Explicit subgoal-chain representations were introduced to address this problem. Rather

than relying only on independent action or route scores, the later architectures relate the current target to intermediate dependencies leading toward the final objective. In Generation II, the program constructs the dependency structure and candidate routes, while the language model scores and selects among those routes. In Generation III, the language model constructs the subgoal chain and proposes a primary target, after which the program uses BFS to verify the reachability of that target. The verifier checks the proposed target, not the logical correctness of the complete model-generated chain. The subgoal-chain representation is intended to keep intermediate dependencies visible, reduce goal drift, and make local decisions easier to interpret as parts of a longer-term plan.

This chapter presents three agent architectures developed as successive generations during the project. They differ in how planning responsibility is divided between the program and the language model, whether an explicit subgoal chain is used, and how programmatic search supports the decision process.

4.3 Generation I: Action Scoring

In the first generation, represented by `mota3_agent25`, the language model does not construct or maintain a subgoal chain. At each decision step, the program performs a coordinate-level breadth-first search on the current floor to identify nearby keys, doors, and stairs. Walls are excluded, and a locked door is excluded when the agent does not currently hold a matching key. The resulting targets are ordered by estimated path length, and at most five are included in the prompt together with their first suggested movement action.

This reachability filtering is intentionally simple. It uses the agent's current key inventory but does not simulate key collection and consumption along the complete path. The candidate targets should therefore be understood as locally plausible targets rather than as globally verified or fully executable routes. By contrast, the immediate actions presented to the model are checked against the current state and are legal at the time of selection.

The language model scores every currently available movement action and recommends exactly one action for execution. It may also associate that action with one of the program-generated candidate targets, but the target is descriptive: only a single movement action is executed before the environment is observed again. The model cannot introduce a target outside the supplied candidate set, although it may select any currently legal action.

This version also includes a basic human supervision channel. Before the recommended action is executed, a human operator may approve it, select another legal action, request an explanation, ask for reconsideration, or terminate the run. When reconsideration is requested, the interaction model may update the scores of the same available actions using the human's feedback.

The prompt states the general long-term objective of reaching a higher floor and also contains recent action history and stored human feedback. However, Agent25 has no explicit representation of the ordered prerequisites leading to the final goal. It does not preserve a structured sequence such as $g_1 \rightarrow g_2 \rightarrow \cdots \rightarrow g^*$ or represent the preconditions $\mathrm{pre}(g_i)$ of later objectives. Decisions remain primarily reactive and are based on locally available actions and targets. This version is therefore used as a reactive baseline for evaluating the later introduction of explicit subgoal structures.

Algorithm 1 Core algorithm of Generation I: Reactive Action Scoring

Observe the current game state s_t.
while the local search has not explored all reachable cells **do**
    Expand the search to each neighbouring cell.
    if the neighbouring cell is a wall, or a locked door without a matching key **then**
        Skip this cell.
    else
        Mark the cell as reachable, and record it as a candidate target if it is a key, door, or staircase.
    end if
end while
Rank all reachable candidate targets by path length, and keep the top-k.
Send the candidate targets and available actions to the LLM.
Let the LLM score the actions and select one action a_t.
Execute a_t in the environment.
Repeat until victory, failure, or manual termination.

4.4 Generation II: Program-Constructed Subgoal Chains

The second generation, represented by `mota5_agent67`, introduces an explicit subgoal-guided planning structure. The planning rules are hand-coded, but the dependency structure is constructed dynamically from the current game state. Starting from the victory condition, the program searches backward through prerequisite doors, required keys, and intermediate objectives, subject to a maximum dependency depth of five.The program uses a Dijkstra-style [15] search to identify blocking doors and uses breadth-first search (BFS) to construct shortest-step executable routes to the resulting targets. These procedures produce a bounded set of candidate routes that are executable from the current state before any route is selected.

The language model does not construct the dependency structure or discover the required doors and keys itself. Instead, it receives the current game state, a summary of the program-generated dependency structure, the candidate routes and their strategic depths, events and byproducts, recent route history, and available human feedback. Its role is to score the candidate routes and select one for execution. If only one route is available, the program selects it directly without querying the language model.

As in Generation I, search is performed internally by the program before the candidate options are presented to the language model. The main difference is that Generation I presents local action or target options, whereas Generation II presents complete multi-step routes. This design guarantees that the presented routes are currently executable, but it does not guarantee that they are globally optimal or that they preserve resources required by later objectives. It is also limited by the fixed dependency depth and by the planner's hand-coded rules. Although the language model can use the supplied state, history, and feedback when scoring routes, it cannot propose a route outside the candidate set generated by the program.

Algorithm 2 Core algorithm of Generation II: Program-Constructed Subgoal Chain

Observe the current game state s_t.
while the dependency tree(subgoal chain) has not reached the maximum expansion depth **do**
    Search backward from the final goal to find the nearest blocking (locked) doors.
    if the target is structurally unreachable **then**
        Discard this branch.
    else
        Record the target, its blocking doors, and whether it is currently reachable.
    end if
    For each blocking door, add the matching key as a new target to expand next.
end while
Generate candidate goals from the dependency tree, prioritising doors that can be opened immediately.
Summarise each candidate route using its steps, events, byproducts, and blocked-door information.
Send the candidate routes to the LLM.
Let the LLM score all routes and choose one route r_t.
Execute r_t in the environment.
Repeat until victory, failure, or manual termination.

4.5 Generation III: LLM-Constructed Chains with Programmatic Verification

The third generation reverses the division of labour established in the previous two generations. Chain construction is done by the language model itself: given a context containing the structured game state and the complete current map state for all floors, the model is prompted to reason about which keys and doors form preconditions of the final goal, and to output a candidate subgoal chain represented as a list of dependent objectives together with a primary target coordinate.

Algorithm 3 Core algorithm of Generation III: LLM-Constructed Subgoal Chain with Programmatic Verification

Observe the current game state s_t and extract structured map objects.
for up to 5 attempts **do**
    Prompt the LLM with the current state, final goal, environment description, and any previous error.
    Let the LLM output a reasoning explanation, a subgoal chain, and a selected target g_t.
    if the LLM output is not valid JSON **then**
        Record the parsing error and try again.
    else
        Run BFS to check whether g_t is reachable from s_t.
        if g_t is reachable **then**
            Execute the path toward g_t in the environment, and stop retrying.
        else
            Record the reachability error and try again.
        end if
    end if
end for
if no valid target was found after 5 attempts **then**
    Abort the run and record it as unsuccessful.
end if
Repeat until victory, failure, or manual termination.

4.6 Mechanisms

Programmatic Verification and Bounded Retry. Breadth-first search is used in all three generations, but its role changes across the agent architectures. In Generations I and II, the program uses search to construct and check candidate targets or routes before presenting them to the language model. The model therefore selects from program-generated candidates rather than proposing an unchecked target. In Generation III, the language model first constructs a subgoal chain and proposes a target. The program then uses a reachability search to check the proposal before execution.
Because a Generation III proposal may contain invalid JSON or an unreachable target, the agent uses a bounded retry loop. The reason for the failed attempt is returned to the model so that it can revise its proposal. If the model does not produce a valid and reachable proposal within the configured retry limit, the planning cycle fails and the run ends unsuccessfully. Chapter 5 describes the implementation of the reachability check and retry process.

Human Supervision. In the interactive mode, a human operator can inspect the model's proposal before it is executed. The operator may approve the proposal, ask a question, request a different proposal, or terminate the run. This supervision can identify a target that is reachable under the current game state but is still a poor choice for the long-term dependency structure. The implementation of this interaction is described in Chapter 5.

Deadlock Detection and Game-State Reset. The advanced Generation III configuration includes an optional soft-deadlock mechanism that runs after each executed route. If the detector identifies the current state as unwinnable, the failed episode is used to generate a lesson and the environment is reset to its initial state s_0. A new episode then begins within the same run. The reset applies only to the environment, while the agent's lesson memory remains available for later decision cycles. The implementation of the deadlock check is described in Chapter 5.

Reflection Memory Across Episodes and Runs. The advanced Generation III configuration can use lessons from two sources. First, lessons saved in logs from previous runs are loaded when the agent starts. Second, when a soft deadlock triggers a reset, the failed episode is summarised into a lesson before the environment is reset. The new lesson is then available to the next episode of the same run. Both sources are stored in a shared, limited lesson memory, and selected lessons are included in later planning prompts. Chapter 5 describes how these lessons are generated, stored, and loaded.

4.7 Summary of Validation and Recovery Mechanisms

The preceding sections introduced five checks and recovery mechanisms used across the Generation III configurations. Each mechanism answers a different question about the model's proposal or the current game state. Table 2 brings them together in a single summary. The soft-deadlock and lesson-extraction mechanisms are used only in the advanced Generation III configuration.

Mechanism	**Question it answers**
LLM output validation	Can the model's response be parsed as JSON, and does it follow the required schema?
Reachability verification	Can the proposed target be reached from the current simulated game state?
Retry limit	Has the model used all configured attempts without producing a valid and reachable proposal?
Soft-deadlock detection (advanced only)	Does the map-specific key–door check classify the current state as unwinnable? If so, should a reset be triggered?
Lesson extraction (advanced only)	What reusable lesson should be generated from the failed episode before the environment is reset?

Table 2: Validation and recovery mechanisms used in Generation III.

4.8 Summary of Architectural Evolution

Table 3 summarises how the three generations differ in three main aspects: which component constructs the subgoal chain, how BFS is used, and whether a failed reachability check causes the language model to revise its proposal.

Generation	Chain construction	Role of BFS	Reachability check	Reachability-guided retry
I: Action scoring	None	Finds reachable local targets	Before LLM selection	No
II: Program-constructed	Hand-coded planner	Constructs and checks candidate routes	Before LLM selection	No
III: LLM-constructed (base)	Language model	Verifies model-proposed targets	After LLM proposal	Yes
III: LLM-constructed (advanced)	Language model	Verifies model-proposed targets	After LLM proposal	Yes

Table 3: Summary of the agent architectures.

All three generations include retries for model-request or output-format errors. However, only Generation III returns reachability errors to the language model and asks it to revise an unreachable proposal.

4.9 Representative Implementation Versions

The table 4 presents several representative agent implementations and indicates the generation to which each one belongs.

Version	Generation	Representative description
`mota3_agent25`	Gen I	**Action scoring.** The program uses BFS to identify a small set of currently reachable candidates. The LLM scores or selects among these local candidates and does not construct an explicit subgoal chain.
`mota5_agent67`	Gen II	**Program-constructed chains.** The program constructs dependency(subgoal) chains or candidate routes in advance. The LLM does not construct the chain itself and only selects among program-provided routes.
`mota5_agent72`	Gen III (base)	**LLM-constructed chains with verification.** The LLM constructs a `subgoal_chain` from the map and final objective. The program then applies BFS-based reachability verification and bounded retries. No historical experience is used.
`mota5_agent70b1`	Gen III (adv)	**LLM-constructed chains with reflection.** This version adds deadlock-triggered reset, historical-experience injection, and an LLM-based reflection mechanism.

Table 4: Representative implementation versions and their corresponding agent architectures.

5 Implementation

5.1 System Architecture and Module Separation

Modularity and separation of responsibilities are important principles in software design [25]. The implementation is therefore organised into three main components: an *engine*, an *agent*, and an *interface*. This separation allows the decision-making logic to change without mixing it with the environment rules or visualisation code. Within the same map configuration, different agent versions use the same engine implementation.

Engine. The engine implements the deterministic transition function T and the tile rules defined in Section 3.3. It provides the current game state and legal actions, applies a requested movement action, and updates the player's position, floor, and key counts. It also opens doors, updates the map, checks the victory condition, and saves the resulting state. The engine contains no strategic decision-making logic: it determines what an action does, but not which action the agent should choose.
Separate engine files are used for the three-floor, five-floor, and seven-floor task configurations because their maps, floor connections, and final goals differ. Although these engine files are not completely identical, they use the same core movement, key, door, staircase, and state-update rules. Agent versions evaluated on the same task configuration share the same engine implementation. This keeps the environment fixed within each controlled comparison.

Agent. The agent implements one of the decision-making architectures described in Chapter 4. It imports the engine module for the corresponding map, loads the current state, constructs a prompt, queries the language model, and processes the returned decision. Depending on the generation, it may also construct candidate routes, verify a model-proposed target, or manage reset and reflection memory. Once an action or route has been selected, the agent calls the engine to execute it.
Keeping the agent logic separate from the engine allows new decision-making versions to reuse the same environment implementation. The agent versions therefore differ mainly in their planning, prompting, verification, and memory mechanisms rather than in the underlying game rules.

Interface. The visual interface is implemented with Pygame. It imports the corresponding engine module and displays the current map, player position, keys, and doors. During an agent run, the interface can monitor the persisted state file and update the display when the state changes. It does not query the language model or control the agent directly. The interface also supports manual play. In this mode, keyboard input is converted into movement actions and sent directly to the engine. The interface may therefore read and update the game state during manual play, while it normally acts as a state viewer during an automated agent run.

Shared State and Communication. The agent and interface both use functions provided by the engine module. The agent communicates with the engine through direct Python function calls, while the engine saves the updated game state in a JSON file after each executed action. The interface monitors the same state file during visual observation.

The persisted file therefore provides a shared representation of the current environment state, but it is not the only communication method between the components.
For unattended experiments, automated agent variants replace manual confirmation with automatic acceptance. This allows the batch scripts to run repeated experiments without waiting for terminal input, while the underlying engine and decision logic remain unchanged.

5.2 Prompt Design and LLM Interface

All representative agent versions in the three generations use the same basic interface to query the language model. The agent sends an HTTP request to a locally or remotely hosted Ollama API endpoint. The request contains the model name, a system prompt, a user prompt, and a fixed set of decoding options.
Two decoding options remain the same in the experiments. First, the request uses the endpoint's JSON output mode. This encourages the model to return valid JSON syntax, but it does not guarantee that the returned object follows the schema required by the agent. Invalid JSON, missing fields, or unexpected values therefore remain possible failure modes.
Second, the sampling temperature is fixed at zero to reduce variation between repeated runs. This does not remove all sources of non-determinism, but it reduces variation caused by random sampling and makes comparisons between agent architectures and prompt designs more controlled.

Prompt content across generations. Although all three generations use the same query method, their prompt content differs substantially. These differences reflect the changing role of the language model in the planning process.
In Generation I, the prompt contains the current state, the map, pre-filtered candidate targets, and the available actions. The model is asked to score the available actions, give a short reason for each score, and recommend one action. It is not asked to construct a subgoal chain or reason about long-term dependencies.
In Generation II, the prompt presents candidate routes that have already been computed by the program. Each route includes its resulting events, byproducts, and blocking relationships. The model is asked to select the best route. It does not construct the dependency chain because this work has already been performed by the program.
In Generation III, the prompt asks the model to perform the dependency reasoning itself. The system prompt contains the environment rules and worked examples of tile-by-tile path checking. The user prompt contains the structured game state and asks the model to return a reasoning explanation, a subgoal chain, and a primary target using a fixed JSON schema. The advanced configuration also asks for a fallback target and includes lessons from previous failures.
When an attempt fails, the next prompt includes the reason for the failure. This may be an output-format or schema-validation error, or a reachability error produced by the verifier. The model can therefore revise its proposal using feedback from the previous attempt instead of trying again without knowing why it failed. The implementation of this retry process is described in the reachability-checking section.

5.3 Programmatic Verification: Reachability Checking

As introduced in Chapter 4, Generation III checks a target proposed by the model before executing any game action. This section explains how the reachability check is implemented.

Searching over full states. A breadth-first search over grid coordinates alone is not sufficient, because reachability depends on the player's position, available keys, and the remaining map configuration. For example, whether the player can pass through a door depends on whether the matching key is available when that door is reached. The verifier therefore performs BFS over simulated game states rather than positions alone.
Starting from the current state, the verifier applies legal actions to generate new states. The search continues until the target floor and position are reached or, for the final target, the victory condition is satisfied. Two states are treated as the same only if their floor, position, key counts, and relevant map configuration are the same. This allows the verifier to distinguish between reaching the same tile with different remaining resources.

Search budget and diagnostics. Searching over full game states produces a much larger search space than searching over grid positions alone. The verifier therefore limits the maximum number of states that may be expanded. If the target is not reached before this limit is reached, the proposal is treated as unreachable for the purpose of the current verification attempt. This does not prove that the target is unreachable without a search limit.
The verifier records the number of expanded states in the run log. A small number usually indicates that the target was easy to verify, while a large number may indicate a longer route, a more complex resource dependency, or a target that cannot be reached in the current state.

Retry after verification failure. If the proposed target cannot be verified, the agent does not execute it. Instead, the reachability error is added to a new prompt and returned to the language model. The model is then asked to produce a revised subgoal chain and target. This process continues until a valid and reachable proposal is produced or the configured retry limit is reached. If the limit is reached, the current planning cycle fails and the run ends unsuccessfully.

Avoiding side effects during search. Verification explores hypothetical states that may never be visited during actual play. These simulated transitions are therefore executed without writing intermediate states to the persistent save file. In this way, verification remains purely exploratory, while only the execution of a verified path updates the real game state.

5.4 Human Supervision

Chapter 4 introduced optional human supervision as a way to inspect and correct a model proposal before execution. This section explains how this interaction is implemented.

Interaction loop. After the model's initial proposal has passed reachability verification, the agent shows the operator the proposed target, the model's reasoning, and the current subgoal chain. The operator can approve the proposal, ask a question, request reconsideration, or terminate the run.
When the operator provides feedback, the agent creates a follow-up prompt containing the current game state, the current proposal, the remaining keys and doors, recent interaction history, and the operator's message. The language model then classifies the message as one of four intents: `answer`, `reconsider`, `execute`, or `quit`.
If the intent is `reconsider`, the model returns a new target, reasoning explanation, and subgoal chain. The program checks the new target with the same BFS-based reachability verifier. If the new target is reachable, it replaces the previous proposal. If it is unreachable, the original verified proposal is kept. Human feedback can therefore change the model's proposal, but it cannot bypass programmatic verification.

5.5 Deadlock Detection and Reflection Memory

Chapter 4 introduced the deadlock-reset and lesson-memory mechanisms used in the advanced Generation III configuration. This section explains how these mechanisms are implemented.

Detecting soft deadlocks. The implementation does not attempt to determine whether every possible game state is unwinnable. Instead, it uses a small, predefined set of critical doors for each map. These doors block the only path to an important area, such as a staircase or a required region on a higher floor.
After each executed route, the detector counts the unopened critical doors of each colour. It then compares this number with the number of matching keys that remain available, including keys currently held and keys remaining on the map. If the number of required critical doors is greater than the number of available keys, the state is classified as unwinnable by this map-specific check.
The detector does not directly help the language model construct its subgoal chain. Instead, it runs after route execution to detect a soft deadlock and trigger a reset. It affects later planning only indirectly, through lessons generated from failed episodes.

Generating a lesson before resetting. When a soft deadlock is detected, the failed episode is not immediately discarded. The implementation creates a structured summary containing the executed route history, the final state, the remaining map objects, and the reason for the deadlock. This summary is sent to the language model through a separate reflection prompt.
The model returns one structured lesson containing the main failure type, a short description of the mistake, a confidence value, and an indication of whether the lesson applies only to the current map or more generally. A lesson is generated only when a deadlock-based reset is triggered. Other unsuccessful endings, such as reaching the cycle limit, do not generate lessons.

Limited lesson storage without duplicates. New lessons are added to an in-memory list. Lessons with duplicate text are discarded, and the list has a fixed maximum size of six by default. When this limit is exceeded, only the most recent lessons are kept. For

each planning prompt, the implementation selects up to the three most recent lessons from this list. It does not use a separate semantic retrieval mechanism.

Resetting the game state. After the lesson has been generated, or skipped because reflection failed, the environment is returned to its initial state and the route history for the failed episode is cleared. The next decision cycle therefore starts from a fresh game state. Only the environment is reset; the lesson memory, conversation history, and operator-feedback memory remain available for later cycles in the same run.

5.6 Logging and Experiment Tracing

Each run produces a readable log and a structured JSON trace. The readable log records the main interaction and execution events in chronological order, including model decisions, BFS results, retries, human input, resets, and reflection lessons. The JSON trace stores one record for each completed decision cycle, including the game state, parsed model decision, selected target, subgoal chain, executed route, and resulting state. The batch evaluation scripts use these records to determine task success, cycle counts, and final states, while runtime and process failures are recorded by the batch runner. These files provide the data used for the qualitative and quantitative analyses in Chapter 7.

Structured trace records. In addition to the readable log, each decision cycle is saved as a structured JSON record in the run's trace file. Each record contains the planning prompt, the parsed model decision, the reachability-check result and number of states expanded, the executed steps and their events, and the updated game state. If a reset occurs, the record also includes the reason for the reset. These trace files provide the main data used for the quantitative analysis in Chapter 7.

6 Experimental Setup

The experiments evaluate nine models accessed through Ollama, including gpt-oss-120b and gpt-oss-20b [26], Gemma 4-31B [27], Qwen3.5-9B and Qwen3.5-397B-A17B [28], Kimi K2.5 [29], GLM-5 [30], MiniMax-M2.7 [31], and DeepSeek-V3.1 [32]. All models were accessed through the same Ollama-based interface, either locally or through Ollama-hosted cloud endpoints [33].

6.1 Experimental Goals

The experiments in this study are designed to answer two related questions: first, whether shifting subgoal-chain construction from a hand-coded planner to the language model (Generation II versus Generation III) changes task reliability; and second, whether adding deadlock detection, game-state reset, and reflection memory to an LLM-constructed-chain architecture (Generation III-base versus Generation III-advanced) improves reliability.
A secondary goal is to examine how these results change when the task is scaled from a five-floor to a seven-floor map. The larger map is used as a proxy for a longer and more complex planning horizon because it contains more floor transitions, resource dependencies, and irreversible decisions. The experiments also compare different language models under the

same agent architecture to examine how model choice affects agent performance. Where runtime cost made this feasible, the same models were evaluated across the representative Generation II and Generation III architectures. Generation I was evaluated separately through qualitative, human-supervised experiments rather than large-scale batch runs.
In addition to these quantitative comparisons, the iterative development process produced two qualitative findings that were retained for analysis. First, early agent versions showed a recurring pattern of goal drift: the model could identify relevant long-term dependencies during interaction but did not consistently maintain the stated intermediate objectives across later decisions. Representative interaction traces of this behaviour are analysed in Chapter 7. Second, experiments with the earliest prompt-driven versions showed that changing the prompt alone could substantially alter agent behaviour while the model and game logic remained unchanged. A comparison of these prompt variants is also presented in Chapter 7. Together, these qualitative analyses complement the quantitative experiments and provide evidence concerning goal maintenance and the influence of prompt design.

6.2 Task Configurations: Floor Count as a Horizon-Length Variable

The main evaluation uses three map configurations: a three-floor configuration (`mota3_*`), a five-floor configuration (`mota5_*`), and a seven-floor configuration (`mota7_*`). These configurations share the same core movement, key, door, staircase, and irreversibility rules. Increasing the number of floors generally introduces more floor transitions, resource dependencies, and irreversible decisions. Floor count is therefore used as a practical proxy for planning-horizon length and task complexity. However, because the maps also differ in layout and dependency structure, floor count should not be interpreted as a perfectly isolated experimental variable.
The evaluation was conducted in stages because long agent runs with cloud-hosted models were expensive. The representative Generation I architecture was studied on the three-floor map, mainly through interactive runs used for the qualitative goal-drift analysis in Section 7.1. A separate one-floor configuration was used for the early prompt-design comparison in Section 7.2. Generation II and the initial Generation III experiments were evaluated on the five-floor map, which provided a moderate task length while still making repeated multi-model batch experiments feasible. The seven-floor map was then used for a smaller comparison of the two Generation III configurations with a reduced set of models. This staged design affects which results can be compared directly. Generation I was not evaluated through the same unattended multi-model batch procedure used for the later generations. On the five-floor map, Generation II (`mota5_agent67`) and the two Generation III configurations (`mota5_agent72` and `mota5_agent70b1`) were evaluated in batch for the cross-agent and cross-model comparison. On the seven-floor map, only the Generation III base and advanced configurations (`mota7_agent72` and `mota7_agent70b1`) were evaluated, using a smaller model set. Table 5 summarises the evaluation coverage of each configuration.

Agent generation	3-floor	5-floor	7-floor
I: Action scoring (`mota3_agent25`)	qualitative only	–	–
II: Program-constructed (`agent67`)	–	9 models	–
III-base: LLM-constructed (`agent72`)	–	6 models	3 models
III-advanced: LLM-constructed + reflection (`agent70b1`)	–	6 models	3 models

Table 5: Experimental coverage by agent generation and map size

6.3 Agent Versions Evaluated

The batch experiments reported in Chapter 7 use one representative implementation for each *evaluated agent configuration*, rather than every intermediate script produced during development. In practice, the five-floor comparison uses `mota5_agent67` for Generation II, `mota5_agent72` for Generation III-base, and `mota5_agent70b1` for Generation III-advanced. The seven-floor confirmatory comparison uses `mota7_agent72` and `mota7_agent70b1`, which extend the two Generation III configurations to the full map. All batch scripts are invoked non-interactively. Human supervision is disabled in these runs, because it requires operator input at each decision point and is therefore incompatible with unattended multi-run evaluation across models. Consequently, the quantitative results in Chapter 7 measure fully autonomous agent performance under each architecture-model pairing.

6.4 Models Evaluated

All models were queried through the same Ollama-compatible HTTP interface, with JSON output mode enabled and the sampling temperature fixed at zero. Models ending in `-cloud` were processed through Ollama's cloud service, which introduced possible service failures unrelated to model reasoning.

Nine models were evaluated with Generation II on the five-floor map. Generation III used the remaining six models after excluding `gemma4:31b-cloud` because of repeated invalid JSON and excluding `glm-5:cloud` and `deepseek-v3.1:671b-cloud` because of repeated service errors.

The seven-floor experiments used a reduced set of three models selected from the earlier five-floor evaluation: `gpt-oss:120b-cloud`, `gpt-oss:20b-cloud`, `qwen3.5:397b-cloud`. Cross-architecture comparisons therefore use only models tested under all relevant configurations.

6.5 Failure Reasons

For every run, the batch runner records whether the episode reached the goal state $\mathcal{G}$, the number of decision cycles elapsed at termination, the wall-clock runtime in seconds, and a failure signal captured directly from the agent process (its exit code, or the exception raised when it terminated abnormally). This raw signal is not, by itself, an explanation: an exit code alone does not distinguish a model that reasoned incorrectly from a cloud endpoint that was temporarily unreachable. Each unsuccessful run was therefore additionally inspected against the tagged communication log and assigned to one of four qualitative failure categories used throughout Chapter 7:

- **Invalid JSON** – the model's response could not be parsed as the expected schema after exhausting the retry budget (e.g. output wrapped in markdown code fences, truncated mid-object, or missing a required field).
- **LLM planning failed** – the response parsed correctly, but every proposed target was rejected by the BFS reachability verifier within the retry budget.
- **Reached cycle limit** – the agent continued to make valid, reachable decisions but did not reach the goal tile within the decision-cycle budget.
- **Service unavailable** (reported in Chapter 7 as "no serve" / "possible model-service overload") – every attempt within a decision cycle failed with an HTTP-level error from the Ollama cloud endpoint (most commonly `410 Gone` or `502 Bad Gateway`) rather than with a malformed or unreachable model response.

7 Results and Discussion

In this section, we present the experimental results, address the research questions, and discuss the main findings and their implications.
The experiments conducted during the development of different agent versions produced the following results: the first generation could only complete the 1-floor Mota task after prompt modifications, while the second generation was already able to complete the 3, 5-floor, tasks.

7.1 Result for RQ1

mota3 agent48 run log

```
[agent_to_human] Step 40: model recommends move right, target blue key at
   (5,3).
[agent_to_human] Current state: M3 (1, 11), keys yellow 0, blue 0, red 0.
[agent_to_human] Step 40 action scores:
[agent_to_human] 1. move_right | score 10.0 | only legal action <-
   recommended
[human_to_agent] <enter>
[agent_to_human] Using the recommended action.
[agent_to_human] Executed action: move_right
[agent_to_human] Result: moved
[agent_to_human] State after action: position 2, 11 on M3, keys yellow 0,
   blue 0, red 0.
[agent_to_human] Step 41: model recommends move right, target blue key at
   (5,3).
[agent_to_human] Current state: M3 (2, 11), keys yellow 0, blue 0, red 0.
[agent_to_human] Step 41 action scores:
[agent_to_human] 1. move_right | score 9.0 | first step of shortest BFS
   path to blue key <- recommended
[agent_to_human] 2. move_left | score 3.0 | leads to downFloor, opposite of
    higherfloor goal
```

[human_to_agent] <enter>
[agent_to_human] Using the recommended action.
[agent_to_human] Executed action: move_right
[agent_to_human] Result: moved
[agent_to_human] State after action: position 3, 11 on M3, keys yellow 0, blue 0, red 0.
[agent_to_human] Step 42: model recommends move left, target downFloor at (1,11).
[agent_to_human] Current state: M3 (3, 11), keys yellow 0, blue 0, red 0.
[agent_to_human] Step 42 action scores:
[agent_to_human] 1. move_left | score 9.0 | matches downFloor target (2 steps), needed to eventually get red key for upFloor <- recommended
[agent_to_human] 2. move_right | score 4.0 | moves away from downFloor; blue key not required now
[human_to_agent] do u think it's better to pick up available key on m3 or m1 now?
[agent_to_human] Right now you have no keys, and the only red key youll ever need is on M1. You can reach M1 by taking the downfloor stairs at (1,11) without picking up any keys on M3 first. The blue key on M3 isnt required for the immediate path, so heading down to M2 (and later to M1) is the better choice.
[human_to_agent] are you sure? can you check the map?
[agent_to_human] Yes, Im sure. The map of floor M3 shows you are currently at (5,11). The downfloor stairs you need are at (1,11) and are reachable by moving left twice, which matches the BFS preview. You have no keys right now, and the only red key youll eventually need to open the red door at (9,11) is on floor M1. So heading left to the downfloor stairs and later going to M1 to pick up the red key is the correct strategy; picking up the blue or yellow keys on M3 isnt required for reaching a higher floor at this moment.
[human_to_agent] If we want to get the red key, can we just move to M1 and get it? Is there no door before the red key?
[agent_to_human] The red key is on floor M1 at (7,6). There is no door blocking you from reaching that tile; you just need to go down from M3 to M2 (using the downfloor stairs at (1,11)) and then further down to M1. Once you are on M1 you can walk to (7,6) and pick up the red key. The red door on M3 (at 9,11) can only be opened after you have that key, but the key itself is freely reachable on M1.
[human_to_agent] ok. go
[agent_to_human] Executing move_left towards downFloor at (1,11).
[agent_to_human] Executed action: move_left
[agent_to_human] Result: moved
[agent_to_human] State after action: position 2, 11 on M3, keys yellow 0, blue 0, red 0.
[agent_to_human] Retrying model decision, attempt 1: answer intent requires current_decision
[agent_to_human] Step 43: model recommends move left, target downFloor at (1,11).
[agent_to_human] Current state: M3 (2, 11), keys yellow 0, blue 0, red 0.
[agent_to_human] Step 43 action scores:

```
[agent_to_human] 1. move_left | score 10.0 | reaches downFloor stairs to
   progress to lower floors <- recommended
[agent_to_human] 2. move_right | score 2.0 | moves away from the target
   stairs
[human_to_agent] <enter>
[agent_to_human] Using the recommended action.
[agent_to_human] Executed action: move_left
[agent_to_human] Result: arrived at M2
[agent_to_human] State after action: position 1, 11 on M2, keys yellow 0,
   blue 0, red 0.
[agent_to_human] Step 44: model recommends move up, target downFloor at
   (1,1).
[agent_to_human] Current state: M2 (1, 11), keys yellow 0, blue 0, red 0.
[agent_to_human] Step 44 action scores:
[agent_to_human] 1. move_up | score 10.0 | only legal action <- recommended
[human_to_agent] <enter>
[agent_to_human] Using the recommended action.
[agent_to_human] Executed action: move_up
[agent_to_human] Result: moved
[agent_to_human] State after action: position 1, 10 on M2, keys yellow 0,
   blue 0, red 0.
[agent_to_human] Step 45: model recommends move down, target upFloor at
   (1,11).
[agent_to_human] Current state: M2 (1, 10), keys yellow 0, blue 0, red 0.
[agent_to_human] Step 45 action scores:
[agent_to_human] 1. move_down | score 9.0 | reaches upFloor leading to
   higher floor <- recommended
[agent_to_human] 2. move_up | score 3.0 | moves toward lower floor
   downFloor
[human_to_agent] aren't we  going to m1 get to red key?
[agent_to_human] Correct  to obtain the red key you need to go to floor M1.
    From your current position (1,10) on M2 the downfloor stairs to M1 are
   at (1,1). The BFS preview shows a path of successive upward moves toward
    that stair. So the next step should be move_up, not move_down.
[human_to_agent] then why you recommend move down
[agent_to_human] I recommended 'move_down' because the BFS preview shows
   that moving down from your current position (1,10) on floor M2 leads
   directly to the upfloor stairs at (1,11). Those stairs take you to floor
    M3, which is a higher floor than M2. Since the overall goal is to reach
    a higher floor, moving down is the most efficient step right now. If
   you want to obtain the red key on M1 first, you would need to head
   toward the downfloor stairs at (1,1) by moving up, but that would take
   you to a lower floor, not the immediate higherfloor goal.
[human_to_agent] Yes, but eventually we need the red key to open the red
   door and reach the upFloor on M3, right?
[agent_to_human] Correct  the red key is required later to open the red
   door on floor M3 (at (9,11)) so you can eventually go up from M3 to an
   even higher floor. However, you can already reach a higher floor right
   now: the upfloor stairs at (1,11) lead from M2 to M3 without any door.
   So the immediate step should be to move down to (1,11) and take those
```

```
   stairs. You can collect the red key on M1 later if you need to open that
    red door.
[human_to_agent] then choose the right way you think now
[agent_to_human] I'll follow the BFS preview and move down to the upfloor
   stairs at (1,11). This takes us to floor M3, which is a higher floor and
    matches the immediate goal.
[agent_to_human] Executed action: move_down
[agent_to_human] Result: arrived at M3
[agent_to_human] State after action: position 1, 11 on M3, keys yellow 0,
   blue 0, red 0.
[agent_to_human] Step 46: model recommends move right, target blue key at
   (5,3).
[agent_to_human] Current state: M3 (1, 11), keys yellow 0, blue 0, red 0.
[agent_to_human] Step 46 action scores:
[agent_to_human] 1. move_right | score 10.0 | only legal action <-
   recommended
[human_to_agent] hello? why you get a conclusion about move down is right
   choice?
[agent_to_human] The decision to move down was based on the BFS preview of
   the current floor (M2). From your position (1,10) on M2, the only
   reachable stair that takes you to a higher floor is the upfloor
   staircase at (1,11). The BFS path shows that moving down (which on this
   floor means stepping onto the tile at (1,11)) leads directly to that
   staircase, allowing you to ascend to floor M3 immediately. While the red
    key on M1 is needed later to open the red door on M3, acquiring it
   first would require moving toward the downfloor stairs at (1,1), which
   would take you to a lower floor (M1) and delay reaching a higher floor
   now. Therefore, moving down was the optimal shortterm step to progress
   upward.
[human_to_agent] while now you have to go back and get red key , am i right
   ?
[agent_to_human] Correct  the red key you need to open the red door on
   floor M3 is located on floor M1. To get it youll have to go back down
   through the stairs: from here (M3 at (1,11)) youll move down to M2, then
    continue down to M1 where the red key is at (7,6). Once you have it you
    can return to M3 and open the red door at (9,11). For now you can keep
   exploring M3 (e.g., head toward the blue key) or start heading back
   whenever youre ready.
[human_to_agent] then choose the correct way
[agent_to_human] Following the BFS preview, the next optimal step is to
   move right toward the blue key at (5,3).
[agent_to_human] Executed action: move_right
[agent_to_human] Result: moved
[agent_to_human] State after action: position 2, 11 on M3, keys yellow 0,
   blue 0, red 0.
[agent_to_human] Step 47: model recommends move right, target blue key at
   (5,3).
[agent_to_human] Current state: M3 (2, 11), keys yellow 0, blue 0, red 0.
[agent_to_human] Step 47 action scores:
```

```
[agent_to_human] 1. move_right | score 9.0 | follows BFS preview toward
   blue key (5,3), the nearest useful item <- recommended
[agent_to_human] 2. move_left | score 4.0 | leads to downFloor and lower
   floors; does not progress toward current reachable target
[human_to_agent] <enter>
[agent_to_human] Using the recommended action.
[agent_to_human] Executed action: move_right
[agent_to_human] Result: moved
[agent_to_human] State after action: position 3, 11 on M3, keys yellow 0,
   blue 0, red 0.
[agent_to_human] Step 48: model recommends move left, target downFloor at
   (1,11).
[agent_to_human] Current state: M3 (3, 11), keys yellow 0, blue 0, red 0.
[agent_to_human] Step 48 action scores:
[agent_to_human] 1. move_left | score 9.0 | Leads toward downFloor (1,11)
   to descend to lower floors and eventually obtain the red key needed for
   the red door. <- recommended
[agent_to_human] 2. move_right | score 4.0 | Continues toward blue key,
   which is not required before acquiring the red key.
[human_to_agent] <enter>
[agent_to_human] Using the recommended action.
[agent_to_human] Executed action: move_left
[agent_to_human] Result: moved
[agent_to_human] State after action: position 2, 11 on M3, keys yellow 0,
   blue 0, red 0.
[agent_to_human] Retrying model decision, attempt 1: answer intent requires
    current_decision
[agent_to_human] Step 49: model recommends move right, target blue key at
   (5,3).
[agent_to_human] Current state: M3 (2, 11), keys yellow 0, blue 0, red 0.
[agent_to_human] Step 49 action scores:
[agent_to_human] 1. move_right | score 7.0 | continues BFS path toward blue
    key <- recommended
[agent_to_human] 2. move_left | score 2.0 | blocked or returns to previous
   tile
[human_to_agent] do u remeber what we talked before?
[agent_to_human] Yes, I remember our discussion. We talked about moving
   down to the stairs at (1,11) to reach a higher floor, the need to obtain
    the red key on floor M1 later to open the red door on M3, and that the
   next immediate BFS suggestion was to move left toward the downfloor
   stairs at (1,11).
[human_to_agent] then why you dont go m1
[agent_to_human] We still need the red key from M1 to open the red door on
   M3, but going there right now would require descending two floors,
   collecting the key, and then climbing back up before we can reach any
   higher floor or the blue key. That would waste many moves while we
   already have easy, reachable items (the blue key) that can be collected
   now and later help us open doors on this floor. So the shortterm optimal
    plan is to keep moving toward the blue key on M3 and deal with the red
   key later when were ready to head down.
```

[human_to_agent] ok, then pick up blue key
[agent_to_human] Moving right toward the blue key at (5,3).
[agent_to_human] Executed action: move_right
[agent_to_human] Result: moved
[agent_to_human] State after action: position 3, 11 on M3, keys yellow 0, blue 0, red 0.
[agent_to_human] Step 50: model recommends move left, target downFloor at (1,11).
[agent_to_human] Current state: M3 (3, 11), keys yellow 0, blue 0, red 0.
[agent_to_human] Step 50 action scores:
[agent_to_human] 1. move_left | score 9.0 | progresses toward downFloor stairs at (1,11) which is reachable <- recommended
[agent_to_human] 2. move_right | score 2.0 | moves away from target and leads to a deadend (up blocked)
[human_to_agent] well, you need to decide we go m1 or pick up blue key
[agent_to_human] From your current position (5,11) the only walkable moves are left or right. Moving up toward the blue key at (5,3) is not possible right now because the Up action is blocked in the available actions list. Going right would hit a wall at (7,11). Therefore the only feasible step is **move_left**, which takes us toward the downstairs at (1,11). Reaching those stairs lets us descend to lower floors and eventually collect the red key on M1, after which we can return and continue toward the blue key. So the next logical move is left.
[human_to_agent] we need go(7,11) to get a blue key? what about bfs told you? what about(6,11)
[agent_to_human] Cell (7,11) is a wall (#) on the map, so it cannot be entered and therefore cannot lead to any key. The only walkable cell to the right of your current position is (6,11), which is empty, but from there the next step right is again a wall, so moving that way would trap you in a deadend. The BFS preview for the blue key at (5,3) shows a path that goes left first (toward the downstairs at (1,11)) and then upward; it never uses (7,11) or (6,11). Hence the sensible immediate move is **move_left**, heading toward the downfloor stairs so you can later descend to M1 for the red key and eventually return for the blue key.
[human_to_agent] what about (5,11)?
[agent_to_human] Cell (5,11) is a plain floor tile ('.'), i.e., walkable empty space. It contains no item, door or stair.
[human_to_agent] isn't this cell a good place to blue key?
[agent_to_human] Cell (5,11) is just an empty floor tile ('.'); there is no blue key there. The blue key you are after is located at (5,3). From your current position (5,11) the only useful move is to go left toward the downstairs at (1,11), which eventually lets you descend to M1 for the red key and later come back for the blue key.
[human_to_agent] check bfs, the route to get blue key from current position
[agent_to_human] From your current position (3,11) the BFS preview for the blue key at (5,3) is:
1. move_right (4,11)
2. move_right (5,11)
3. move_up (5,10)

4. move_up (5,9)
5. move_up (5,8)
6. move_up (5,7)
The preview shows the first six actions; the remaining four moves are continue moving up from (5,7) to (5,6), (5,5), (5,4) and finally (5,3) where the blue key is located. Total steps = 10.
[human_to_agent] yes , now you konow how to get a blue key right?
[agent_to_human] Yes, I know the route to the blue key. From your current position (4,11) the BFS preview shows: move_right (5,11), then move_up repeatedly until reaching (5,3) where the blue key is located.
[human_to_agent] go get it
[agent_to_human] Okay, moving right towards the blue key.
[agent_to_human] Executed action: move_right
[agent_to_human] Result: moved
[agent_to_human] State after action: position 4, 11 on M3, keys yellow 0, blue 0, red 0.
[agent_to_human] Step 51: model recommends move left, target downFloor at (1,11).
[agent_to_human] Current state: M3 (4, 11), keys yellow 0, blue 0, red 0.
[agent_to_human] Step 51 action scores:
[agent_to_human] 1. move_left | score 8.0 | Advances toward downFloor (1,11) needed for red key and later blue key. <- recommended
[agent_to_human] 2. move_right | score 2.0 | Would step into wall or deadend, no useful progress.
[human_to_agent] why you still care about red key right now? aren't we are going to pick up blue key? do you still remeber what we talked seconds ago
[agent_to_human] I remember our last discussion: we were heading for the blue key at (5,3). However, to eventually leave this floor you must reach the upstairs at (11,11). That stair is blocked by a red door at (9,11), and the only red key available is on floor M1. So even while we are on the way to the blue key, we still need to plan for getting the red key later; otherwise well be stuck after the blue key. The immediate next step is still to move left toward the downstairs at (1,11) so we can descend, collect the red key on M1, and then return for the blue key .
[human_to_agent]Do you still remember that we chose to pick up the red key on M1 and then return to pick up the blue key?
[agent_to_human] Yes, I remember our plan: we first go down to M1 to pick up the red key, then come back to this floor to collect the blue key at (5,3). Until we have the red key we cannot exit via the upstairs, so heading toward the downstairs at (1,11) is still the right move.
[human_to_agent] no, we are going to pick up blue key
[agent_to_human] I understand you want to go for the blue key at (5,3). The BFS preview for that key starts with a move_right from (5,11) followed by several move_up actions. However, the current available actions list shows that moving up is blocked, so we cannot start the aupward part of the path right now. The only walkable moves are left or right. Moving right would put us at (6,11), which is a deadend (the next cell to the right is a wall). Moving left takes us toward the downstairs at (1,11);

```
   from there we can descend, collect the red key on M1, and later return
   to continue upward to the blue key. So the next feasible action is **
   move_left**.
[human_to_agent] no, pick up blue key
[agent_to_human] From your current position (5,11) moving up toward the
   blue key is blocked (the game reports move_up as unavailable). The only
   walkable moves are left or right. Moving right would put you at (6,11)
   and then hit a wall, leading to a deadend. Moving left heads toward the
   downstairs at (1,11), which is the only way to leave this floor, get the
    red key on M1, and later return for the blue key. So we cannot pick up
   the blue key right now; the next feasible step is **move_left**.
[human_to_agent] ok, do what u want until you get blue key or red key ,i
   give you step like 50
[agent_to_human] Proceeding left toward the downstairs at (1,11) so we can
   descend, pick up the red key on M1 and later return for the blue key.
[agent_to_human] Executed action: move_left
[agent_to_human] Result: moved
[agent_to_human] State after action: position 3, 11 on M3, keys yellow 0,
   blue 0, red 0.
[agent_to_human] Step 52: model recommends move left, target downFloor at
   (1,11).
[agent_to_human] Current state: M3 (3, 11), keys yellow 0, blue 0, red 0.
[agent_to_human] Step 52 action scores:
[agent_to_human] 1. move_left | score 9.0 | Leads to downFloor at (1,11),
   required to descend, get the red key and eventually exit the floor. <-
   recommended
[agent_to_human] 2. move_right | score 2.0 | Moves toward a deadend; cannot
    advance toward any key or stair currently.
[human_to_agent] <enter>
[agent_to_human] Using the recommended action.
[agent_to_human] Executed action: move_left
[agent_to_human] Result: moved
[agent_to_human] State after action: position 2, 11 on M3, keys yellow 0,
   blue 0, red 0.
[agent_to_human] Retrying model decision, attempt 1: answer intent requires
    current_decision
[agent_to_human] Step 53: model recommends move right, target blue key at
   (5,3).
[agent_to_human] Current state: M3 (2, 11), keys yellow 0, blue 0, red 0.
[agent_to_human] Step 53 action scores:
[agent_to_human] 1. move_right | score 9.0 | advances toward blue key
   target <- recommended
[agent_to_human] 2. move_left | score 1.0 | blocked wall at map edge
[human_to_agent] no, we are going to pick up red key
[agent_to_human] Got it, we'll head for the red key. The next step is
   moving left onto the downstairs at (1,11) to go down a floor.
[agent_to_human] Executed action: move_left
[agent_to_human] Result: arrived at M2
[agent_to_human] State after action: position 1, 11 on M2, keys yellow 0,
   blue 0, red 0.
```

Discussion

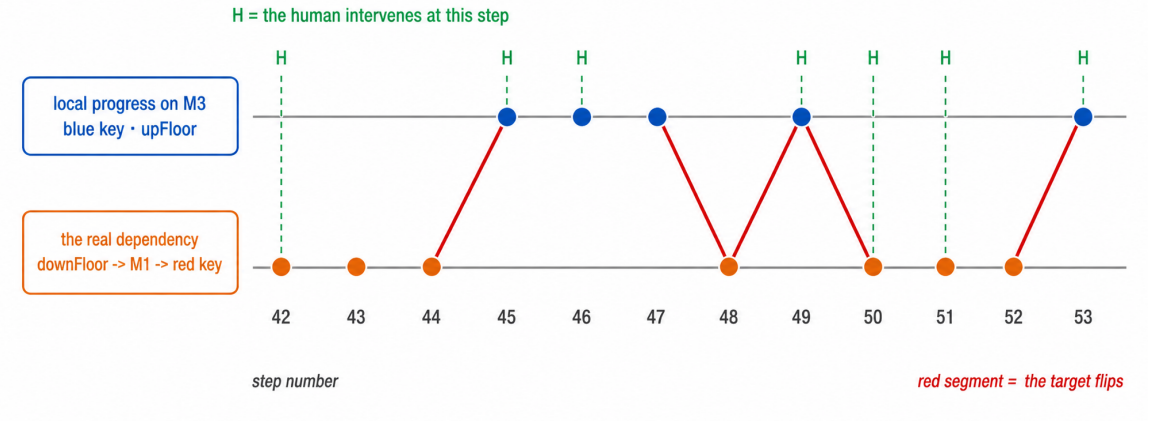


Figure 4: Line chart for goal drift analysis.

Step-by-step evidence of goal drift (Steps 42–53). The interaction log shows that the model failed to maintain a stable subgoal sequence and repeatedly switched between locally available objectives and longer-term resource dependencies.

At **Step 42**, the model abandoned the previously active direction on M3 and redirected the agent toward the `downFloor at (1,11)`. The immediate objective changed from continuing local progress on M3 to returning to M1 in order to collect the `red key`.

At **Step 43**, after the human asked(step 42) whether it would be better to collect an available key on M3 or to go to M1 first, the model reinforced the new staircase-oriented plan. It argued that the red key on M1 was more important than the currently reachable resources on M3 and continued to prioritize movement toward the staircase.

At **Step 44**, the agent continued to follow this route. The decision process remained focused on descending toward M1 rather than exploiting the immediately reachable opportunities on M3.

At **Step 45**, however, once the agent reached M2, the model abruptly reversed its plan. Instead of continuing toward M1 for the red key, it recommended moving toward the `upFloor at (1,11)` and returning to M3. This was the first clear reversal: the model had just argued that descending for the red key was the correct priority, but it now preferred immediate upward progress again.

At **Step 46**, the model acknowledged that the red key would still be required later, but nevertheless kept the new short-term preference for returning to M3. This shows that the model could verbally preserve the long-term dependency while failing to keep it as the active decision constraint.

At **Steps 47–49**, after returning to M3, the model reoriented its behavior around local resources on that floor and shifted back toward the `blue key`. At this point, the previously emphasized red-key route was no longer operationally maintained.

At **Step 50**, after only one move in the direction of the blue key, the model switched once more. It returned to the `downFloor at (1,11)` plan and again framed the next action in terms of descending toward M1.

At **Step 51**, the human reminded the model that the chosen objective was to pick up the blue key, and BFS also provided a concrete route. The model stated that it understood this route, but still reframed the next feasible action as part of the red-key and staircase plan. This is a particularly clear example of instability: the model recognized the local route but did not sustain that route as the active goal.
At **Step 52**, the model continued to execute the return-to-M1 direction. The active objective once again became the staircase and the future acquisition of the red key.
At **Step 53**, shortly afterward, the model shifted attention again toward progress on M3, including the blue-key direction. This marked another reversal and showed that the agent was not following a stable subgoal chain, but oscillating between competing short-term and long-term targets.
Taken together, Steps 42–53 provide qualitative evidence of goal drift: although the model could articulate why a long-term dependency mattered, it repeatedly failed to keep that dependency stably connected to the next action decision.

7.2 Result for RQ2

Agent13 and Agent14 are earlier versions from the development process. They were used only to test the effect of prompt design.The figure below is used to analyse the log.The coordinates start from the upper-left corner, with indexing beginning at zero.

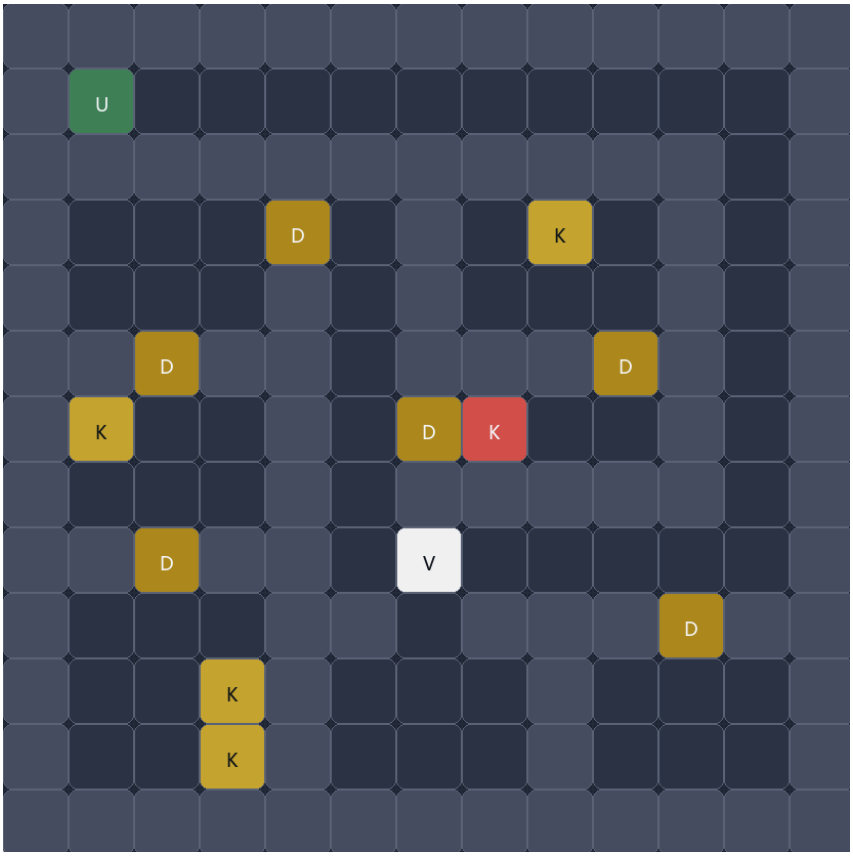


Figure 5: M1 floor.

Agent13 Log

```
New game started. Current position: M1 (6, 11), keys yellow 0.
Step 1: stage collect_key, choose to move up, target Yellow Key at (5,10).
Current state: M1 (6, 11), keys yellow 0.
Result: moved
State after action: position 6, 10 on M1, keys yellow 0.

Step 2: stage collect_key, choose to move left, target Yellow Key at (5,10)
    .
Current state: M1 (6, 10), keys yellow 0.
Result: got yellow key
State after action: position 5, 10 on M1, keys yellow 1.

Step 3: stage push_stairs, choose to move right, target Yellow Door at
    (6,9).
Current state: M1 (5, 10), keys yellow 1.
Result: moved
State after action: position 6, 10 on M1, keys yellow 1.

Step 4: stage push_stairs, choose to move up, target Yellow Door at (6,9).
Current state: M1 (6, 10), keys yellow 1.
Result: opened yellow door
State after action: position 6, 9 on M1, keys yellow 0.

Step 5: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (6, 9), keys yellow 0.
Result: moved
State after action: position 6, 8 on M1, keys yellow 0.

Step 6: stage push_stairs, choose to move left, target Stairs at (1,1).
Current state: M1 (6, 8), keys yellow 0.
Result: moved
State after action: position 5, 8 on M1, keys yellow 0.
```

Agent14 Log

```
New game started. Current position: M1 (6, 11), keys yellow 0.
Step 1: stage collect_key, choose to move up, target Yellow Key at (5,10).
Current state: M1 (6, 11), keys yellow 0.
Result: moved
State after action: position 6, 10 on M1, keys yellow 0.

Step 2: stage collect_key, choose to move left, target Yellow Key at (5,10)
    .
Current state: M1 (6, 10), keys yellow 0.
Result: got yellow key
State after action: position 5, 10 on M1, keys yellow 1.
```

```
Step 3: stage push_stairs, choose to move right, target Yellow Door at
   (6,9).
Current state: M1 (5, 10), keys yellow 1.
Result: moved
State after action: position 6, 10 on M1, keys yellow 1.

Step 4: stage push_stairs, choose to move up, target Yellow Door at (6,9).
Current state: M1 (6, 10), keys yellow 1.
Result: opened yellow door
State after action: position 6, 9 on M1, keys yellow 0.

Step 5: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (6, 9), keys yellow 0.
Result: moved
State after action: position 6, 8 on M1, keys yellow 0.

Step 6: stage push_stairs, choose to move right, target Stairs at (1,1).
Current state: M1 (6, 8), keys yellow 0.
Result: moved
State after action: position 7, 8 on M1, keys yellow 0.
```

The full logs are provided in the appendix.

Controlled variable. agent13 and agent14 are same code except for the system prompt: agent14 adds a small set of targeted consistency constraints requiring the chosen action to match the target's `path_start` field and prioritizing "consistency between target and action over vague exploration." No game logic, model, or decoding parameters changed (both runs used `gpt-oss:120b`).

Steps 1–5: identical in both runs. Both agents collect the yellow key, backtrack to unlock the door at $(6, 9)$, and arrive at $(6, 8)$. Until this point, both agents behave the same.

Step 6 — the fork. At $(6, 8)$, the environment already supplies the correct next move as reliable, engine-computed shortest-path guidance — `.path_start = "move_right"`, the first action of a breadth-first-search shortest path from the agent's current position to the selected candidate target. The two models diverge on identical input:

- **Agent13's** raw `thinking` field: *"From current (6,8), moving left seems good to go toward column 1... Possibly left is best immediate."* It substitutes a naive heuristic (the destination is at $x = 1$, so decrease x) for the supplied `path_start`[1], and never references `path_start` at all.
- **Agent14's** raw `thinking` field: *"Candidate target is stairs at (1,1) with path_start 'move_right'. So ideally we should move right if available, matching path_start."* It cites the prompt's new instruction and follows the guidance after confirming the cell is open.

[1]This field contains the first action of a BFS shortest path from the players current position to the corresponding candidate target. It is computed by the code rather than generated by the LLM.

Consequence of the fork. Moving left from position $(6, 8)$ enters a one-tile-wide vertical corridor at map column $x = 5$. The corridor terminates at $(5, 3)$ because the tile at $(5, 2)$ is blocked; row $y = 2$ is otherwise passable only at $x = 11$. Agent13 enters this dead-end corridor and repeatedly moves between $y = 3$ and $y = 6$. The same behaviour is reproduced in both independent runs: each continues for more than 30 steps, with the agent repeatedly moving within the corridor but never attempting the `left` or `right` actions needed to escape. After choosing the correct path, Agent14 moves steadily: five steps right, six steps up, and nine steps left. It reaches the stairs in 27 steps without visiting any cell twice.
The complete run logs for both agents, together with the decision-trace excerpts at Step 6 shown above, are provided in Appendix B for readers who wish to verify the complete step-by-step context underlying this comparison.

Discussion

1. **A small prompt change led to very different outcomes.**The environment, model, and initial game state were kept the same. The only intended change was the addition of several prompt instructions. Agent14 completed the task, whereas Agent13 became trapped in a corridor and eventually failed. This comparison suggests that small changes in prompt instructions can strongly affect the agent's decisions.

2. **The main difference was one instruction that changed which information the model followed.** The baseline prompt allowed the model to use a simple spatial rule, such as moving toward the target coordinates, when no stronger guidance was available. The revised prompt did not add new state information: both agents already received `path_start`. Instead, it told the model to follow this field before using its own spatial heuristic. This suggests that failures in long-horizon tasks are not always caused by missing information. They can also result from giving the wrong priority to information that is already available. Thus, prompt design can help decide between competing signals, rather than simply provide new information.

3. **Failure at a branch point is catastrophic and irreversible without explicit backtracking guidance.** Agent13 made one wrong turn at step 6 of a run that lasted more than 27 steps. The model's decisions were greedy[6] and local: at each step, it reconsidered moving "up" or "down" based on nearby walls, but it never reconsidered whether it should leave the current branch. This suggests that the model needs an explicit rule explaining how to use the available context. This is the main instruction added in v14.

4. **The agent repeatedly moved within a small area.** Agent13 moved between three or four cells and repeatedly chose a direction that led back toward the wall it had just reached. This behaviour shows that the agent focused on the local situation rather than considering the larger route. When it reached the end of the corridor, it kept repeating the same type of move instead of reconsidering the route.

Key prompt additions in `mota1_agent14`

Compared with `mota1_agent13`, `mota1_agent14` added the following consistency constraints:

```
If you choose a target that has a path_start, your action should usually
match that path_start.
Do not say you are targeting stairs, keys, or doors and then move in a
different direction unless the path_start is unavailable.
When a target is selected, prioritize consistency between target and action
over vague exploration.
```

7.3 Cross-Model Comparison on the Five-Floor Task

Because the five-floor map has a moderate level of dependency complexity, it provides a useful setting for comparing agent architectures and language models. The task configuration, map, and success condition remain fixed across the three tables. Within each table, the model is varied while the agent architecture is kept fixed. Across the tables, the agent architecture changes from the program-guided Generation II configuration to the two LLM-planning-based Generation III configurations. In Generation II, the hand-coded planner constructs dependency-consistent candidate routes and the language model selects among them. In Generation III, the language model constructs a subgoal chain, after which the program checks the proposed target using BFS-based reachability verification. Nine models were evaluated with Generation II. The six models that produced at least one parseable response during that stage were carried forward to both Generation III configurations.

Model	Runs	Successes	Success Rate	Avg Runtime (s)	Avg Cycles to Success	Main Failure Reason
gpt-oss:120b-cloud	5	5	100%	643.89	5.00	–
gpt-oss:20b-cloud	5	5	100%	702.47	5.00	–
gemma4:31b-cloud	5	0	0%	2074.72	–	invalid JSON
qwen3.5:9b	5	5	100%	845.92	5.00	–
qwen3.5:397b-cloud	5	5	100%	647.56	5.00	–
kimi-k2.5:cloud	5	1	20%	725.57	5.00	invalid JSON
glm-5:cloud	5	0	0%	93.21	–	no serve
minimax-m2.7:cloud	5	5	100%	638.16	5.00	–
deepseek-v3.1:671b-cloud	5	0	0%	59.74	–	no serve

Table 6: Model comparison on the 5-floor Mota task by agent67

Model	Runs	Successes	Success Rate	Avg Runtime (s)	Avg Cycles to Success	Main Failure Reason
gpt-oss:120b-cloud	5	3	60%	3339.31	12.33	LLM planning failed
gpt-oss:20b-cloud	1	0	0%	32383.62	–	LLM planning failed
qwen3.5:9b	5	0	0%	3799.46	–	invalid JSON
qwen3.5:397b-cloud	5	1	20%	4966.65	15.00	Reached cycle limit
kimi-k2.5:cloud	5	0	0%	365.42	–	invalid JSON
minimax-m2.7:cloud	5	0	0%	2185.44	–	LLM planning failed

Table 7: Model comparison on the 5-floor Mota task by agent72

Generation II: a program-guided baseline. Generation II achieved the highest success rates among the three configurations. Five of the nine tested models completed all five runs, and successful runs generally finished in exactly five decision cycles.

Model	Runs	Successes	Success Rate	Avg Runtime (s)	Avg Cycles to Success	Main Failure Reason
gpt-oss:120b-cloud	5	3	60%	5398.23	15.67	LLM planning failed
gpt-oss:20b-cloud	5	0	0%	8839.00	–	LLM planning failed
qwen3.5:9b	5	0	0%	5737.91	–	invalid JSON
qwen3.5:397b-cloud	5	1	20%	6430.32	24.00	LLM planning failed
kimi-k2.5:cloud	5	0	0%	526.98	–	invalid JSON
minimax-m2.7:cloud	5	0	0%	999.36	–	invalid JSON

Table 8: Model comparison on the 5-floor Mota task by agent70b1

This pattern is consistent with the design of `mota5_agent67`: the hand-coded planner has already generated dependency-consistent candidate routes before the language model is queried.
The unsuccessful or partially successful models mainly failed for reasons unrelated to long-horizon dependency construction. `gemma4:31b-cloud` repeatedly returned markdown-fenced or truncated JSON that the parser rejected. `kimi-k2.5:cloud` completed only one of five runs and failed in the remaining runs because of invalid JSON responses. `glm-5:cloud` and `deepseek-v3.1:671b-cloud` did not provide a usable service response during the runs. Therefore, the Generation II results measure to a large extent whether a model can return a parseable structured response and select from program-generated candidates, rather than whether it can independently construct a long-horizon plan.

Generation III: a substantially harder planning task. Performance decreases sharply when the language model is required to construct its own dependency chain. In Agent72, the model proposes both the chain and the next target, while the program only verifies the proposal after it has been generated. On the five-floor task, `gpt-oss:120b-cloud` achieved a 60% success rate and `qwen3.5:397b-cloud` achieved a 20% success rate. The other tested models achieved no successful runs.

Generation III-advanced: reset and reflection mechanisms. Agent70b1 extends the Agent72 architecture with deadlock-triggered reset and reflection-based lesson storage.
On the five-floor task, it did not produce a clear improvement in success rate. For `gpt-oss:120b-cloud`, both Agent72 and Agent70b1 achieved a 60% success rate. For `qwen3.5:397b-cloud`, both achieved a 20% success rate.
Agent70b1 nevertheless required more runtime and, for the successful runs, more decision cycles on average.

Discussion

The results show a clearer relationship between model capability and success when more planning responsibility is given to the language model. In Agent67, the program constructs dependency-consistent routes before the model is queried, and several models therefore achieve high success rates.
In contrast, Agent72 and Agent70b1 require the model to construct and maintain its own long-horizon subgoal chain. Under these conditions, the stronger models perform better: `gpt-oss:120b-cloud` achieves the highest success rate, while `qwen3.5:397b-cloud` shows only limited success and the weaker tested models generally fail.
The language model can provide flexible reasoning, but the experiments indicate that it

is currently less efficient and less reliable than a procedural planner when the task can be solved using clear algorithmic structure.
If stronger models generally perform better, why the larger `qwen3.5:397b-cloud` model performed worse than `gpt-oss:120b-cloud`? There are at least two possible explanations. First, the Qwen 397b model worse than gpt120b model, despite having more parameters. Second, the GPT model may be better suited to this agent architecture, especially its prompting format, structured-output requirements, and long-horizon planning process.
These results do not show that reflection is ineffective in general. Rather, the five-floor experiments do not isolate a clear benefit from the additional mechanisms. Resetting, reflection, and the associated replanning introduce additional processing, while the task may not be difficult enough for their potential benefits to outweigh this overhead.

7.4 Cross-Model Comparison on the Seven-Floor Task

Model	Runs	Successes	Success Rate	Avg Runtime (s)	Avg Cycles to Success	Main Failure Reason
gpt-oss:120b-cloud	5	4	80%	8281.60	24.25	Reached cycle limit
gpt-oss:20b-cloud	5	0	0%	11852.93	–	Possible model-service overload
qwen3.5:397b-cloud	5	0	0%	0.29	–	Possible model-service overload

Table 9: Model comparison on the 7-floor Mota task by agent72

Model	Runs	Successes	Success Rate	Avg Runtime (s)	Avg Cycles to Success	Main Failure Reason
gpt-oss:120b-cloud	5	5	100%	3940.57	22.00	–
gpt-oss:20b-cloud	5	1	20%	12313.94	27.00	Possible model-service overload
qwen3.5:397b-cloud	5	0	0%	0.22	–	Possible model-service overload

Table 10: Model comparison on the 7-floor Mota task by agent70b1

The 7-floor results show a clearer separation between agent architectures than the 5-floor task. As task depth increases, the limitations of the non-reflective Gen III base version become more visible, while the reflection-enabled variant begins to show practical advantages. In particular, `Agent70b1` achieved stronger overall performance than `Agent72` on the 7-floor task. For `gpt-oss:120b-cloud`, `Agent72` reached 4/5 successes, whereas `Agent70b1` achieved 5/5 successes with a lower average runtime and a lower average cycle count to success.
This indicates that under a more demanding long-horizon setting, the additional reset and reflection mechanisms can improve both robustness and efficiency rather than merely adding overhead.
Overall, the 7-floor experiments support a stronger conclusion than the 5-floor task. On simpler maps, reflection-style mechanisms do not necessarily improve results and may even introduce extra overhead. However, once the task becomes sufficiently long-horizon and failure recovery becomes more important, the reflection-enabled architecture begins to show measurable benefits. This suggests that the value of reflection is not universal, but difficulty-dependent: it becomes most useful when the planning horizon is long enough that recovery, replanning, and limited self-correction matter more than raw local decision speed.

7.5 Why did the weaker 20B model fail on the 5-floor task, yet achieve one success on the 7-floor task under Agent70b1?

At first sight, the results for `gpt-oss:20b-cloud` may appear to suggest that the reflection-enabled `Agent70b1` architecture provides a meaningful advantage. On the 7-floor task, `Agent72` achieved no successful runs, whereas `Agent70b1` achieved one success out of five. However, a closer reading of the run logs shows that this interpretation would be too strong. The evidence does not show a clear case in which reflection directly rescued a failing trajectory. Instead, the logs suggest that the 5-floor and 7-floor failures differ in type, and that the single 7-floor success was mainly the result of a sufficiently stable early trajectory rather than a reset.

5-floor task: failures are dominated by early planning breakdowns. On the 5-floor task, the main bottleneck for `gpt-oss:20b-cloud` is not late recovery but unstable proposal generation at the planning stage itself. In the failed runs, the model repeatedly produced targets that were later rejected by BFS as unreachable. For example, in one representative run, the model proposed first collecting the yellow key at `M2(3,4)` and later the yellow key at `M1(8,3)`, but both targets were rejected by the verifier:

Representative 5-floor failure: unreachable targets

```
[agent_to_human]   BFS expanded 76318 states
[agent_to_human] BFS failed for primary target:
BFS could not reach collect yellow key at M2 (3,4)
(floor=M2, x=3, y=4).

...

[llm_to_agent] target
collect yellow key at M1 (8,3)

[agent_to_human]   BFS expanded 76318 states
[agent_to_human] BFS failed for primary target:
BFS could not reach collect yellow key at M1 (8,3)
(floor=M1, x=8, y=3).

[agent_to_human] LLM planning failed:
LLM failed to produce a reachable target after 5 attempts.
```

This pattern indicates that the model was often unable to produce a valid and executable next subgoal even before any deadlock-reset or reflection mechanism could become relevant. In other words, the dominant failure mode on the 5-floor task was front-loaded planning unreliability rather than a recoverable long-horizon mistake.

7-floor task: failures are more varied. The 7-floor failures are more varied. Some runs still failed in ways similar to the 5-floor task, including invalid or unreachable proposals and unstable outputs. For example, one failed run combined an unreachable target with subsequent backend errors:

Representative 7-floor failure: unreachable proposal followed by backend failure

```
[llm_to_agent] subgoal_chain
  current position (5,10) -> reachable area -> find reachable keys
  only reachable key is red at (6,7)

[agent_to_human]   BFS expanded 76318 states
[agent_to_human] BFS failed for primary target:
BFS could not reach Collect the red key at M1 (6,7)
(floor=M1, x=6, y=7).

[agent_to_human] LLM output invalid (attempt 2):
Cannot reach Ollama at http://127.0.0.1:11434/api/generate:
HTTP Error 500: Internal Server Error
```

Other failed runs were dominated by repeated request failures, timeouts, or non-JSON outputs, again preventing the model from sustaining a usable plan long enough for recovery mechanisms to matter.

7-floor task: resource-lock detection does occur, but it does not clearly rescue a run. Unlike the 5-floor runs, the 7-floor logs also contain failure cases that are more directly relevant to the reset-and-reflection design. In one run, the system detected a soft deadlock caused by insufficient remaining blue keys:

Representative 7-floor reset trigger

```
[agent_to_human] Auto-reset triggered:
unwinnable resource lock detected:
3 critical blue door(s) remain at M4(7,1), M4(8,1), M6(5,1),
but only 2 total blue key(s) remain (inventory + map pickups).

[agent_to_human]   Lesson from mota7_agent70b1_auto_run_004.log:
Reserve scarce keys for critical doors;
collect all required keys before using any.
```

This is qualitatively different from the 5-floor planning collapses, because here the run progressed far enough for the agent to identify a genuine resource-budget error and to generate a lesson from it. Nevertheless, this run still did not succeed afterward. Thus, while the reset and reflection machinery was activated, the logs do not show that it converted the run into a successful one.

The single successful 7-floor run did not depend on reset. The only successful 7-floor run for `gpt-oss:20b-cloud` under `Agent70b1` does not provide evidence that reset or reflection directly rescued a failing trajectory. In that run, no reset was triggered. Instead, the model happened to maintain a sufficiently consistent sequence of executable subgoals from relatively early in the episode. The run advanced through staircase progression, reached `M6`, then correctly selected the red key on `M7`, and finally moved to the exit:

Representative 7-floor success without reset

```
[agent_to_human]   State: M5 (10,10)  keys yellow 3, blue 0, red 0

=== Cycle 25 ============================================
[llm_to_agent] target
Move to the up-stairs at (11,11) on M5 to ascend to M6

[agent_to_human]   BFS expanded 6 states
[agent_to_human]   Result: arrived at M6

=== Cycle 26 ============================================
[llm_to_agent] target
Collect the red key at M7(6,3) to open the red door at M7(11,6).

[agent_to_human]   BFS expanded 994 states
[agent_to_human]   Result: got red key

=== Cycle 27 ============================================
[llm_to_agent] target
Move to the exit staircase at (11,11) on M7

[agent_to_human]   BFS expanded 71 states
[agent_to_human]   Result: reached final stairs
[agent_to_human] Victory reached!
```

This suggests that the successful run was mainly the result of an unusually stable early decision sequence, which the verification layer was able to keep on track, rather than a case in which reflection actively corrected an already failing trajectory.

Taken together, the logs support a more cautious interpretation of the 20B results. On the 5-floor task, failures are dominated by early planning unreliability, where reflection has little opportunity to help. On the 7-floor task, the agent does sometimes encounter resource-lock states that are, in principle, appropriate targets for reset and reflection. However, the only successful run does not appear to depend on such recovery. Therefore, the current evidence does not justify claiming that reflection itself was the decisive factor behind the 7-floor success of `gpt-oss:20b-cloud`. A more defensible conclusion is that the broader `Agent70b1` framework can occasionally sustain a viable trajectory for an otherwise fragile model, but the logs do not show a clear case in which reflection directly transformed a failing run into a successful one.

8 Conclusion

This section summarizes how the experimental results answer the research questions introduced in Chapter 1.

RQ1: What kinds of long-horizon decision failures occur in LLM agents under irreversible resource constraints? The results for RQ1 show that agents that score actions at each turn can suffer from goal drift. This can happen even when the LLM remembers the correct order of the required dependencies. The main problem is not missing information. Rather, the agent lacks a persistent way to keep following a correct plan in

later decisions. As a result, a target that looks useful locally can replace a target that is necessary for the overall task. These results support using an explicit subgoal chain to preserve dependencies and keep the agent aligned with the final goal over many decisions.

RQ2:How does prompt design influence the decision-making behavior of LLM agents in long-horizon tasks? Prompt design can be the deciding factor between full task success and total failure on a long-horizon task, even when the model and environment are held fixed. The critical intervention was not the generality or verbosity of the prompt, but a targeted instruction forcing the agent to align its chosen action with an environment-supplied path hint that it would otherwise override with an unverified coordinate-direction inference.

RQ3: Does explicit subgoal-chain reasoning reduce goal drift and resource–planning errors? In Generation II, the program constructed a dependency tree that functioned as an explicit subgoal chain. This enabled the agent to complete tasks that the earlier Generation I agents could not solve. In Generation III, responsibility for constructing the explicit subgoal chain was transferred to the LLM. Although both runtime and the average number of cycles to success increased sharply compared with Generation II, some Generation III configurations were still able to complete the multi-floor tasks, whereas the earlier Generation I versions demonstrated success only on the single-floor task.
The results also suggest that goal drift became less frequent after explicit subgoal chains were introduced. However, the experiments do not show that explicit subgoal-chain reasoning consistently reduces resource-planning errors. Weaker models may still generate incorrect subgoal chains, meaning that the agent can produce a more detailed plan that nevertheless remains invalid.

RQ4: Can programmatic verification and experience reflection improve the reliability of LLM agents in resource-constrained environments? The experiments show that programmatic verification is consistently useful, while the benefit of reflection is more limited and depends on the task. BFS-based verification can reliably reject unreachable targets and prevent some invalid model suggestions from being executed. Resource-lock detection also improves reliability by identifying unwinnable states that the environment itself does not directly report as failures. Reflection-based mechanisms, including resets and lesson extraction, show less consistent results. On the 5-floor task, they did not clearly improve performance over the simpler Gen III baseline. On the 7-floor task, however, the reflection-enabled architecture showed some improvement, such as increasing the success rate of `gpt-oss:120b-cloud` from 4/5 to 5/5. At the same time, not every successful run directly depended on reset or reflection. Therefore, the results show that verification clearly improves reliability, while reflection can be useful in harder long-horizon tasks but does not always play a decisive role.

8.1 Future work

First, the current experiments use simplified Mota environments with limited map size and dependency complexity. Future work should evaluate the agent architectures on larger maps, deeper floor structures, more varied key–door dependencies, and partially observable environments. This would test whether the observed behaviour of subgoal-chain

reasoning and target-reachability verification remains consistent as the planning horizon becomes longer. Finally, the framework could be developed into a broader testbed for interpretable LLM-agent decision making. Because communication between the game engine, agent, and language model is recorded, future work could develop automatic measures of goal drift, resource-planning errors, subgoal consistency, and replanning quality. These measures would support more systematic comparisons of prompts, models, and agent architectures.

References

[1] G. Zhang, H. Geng, X. Yu, Z. Yin, Z. Zhang, Z. Tan, H. Zhou, Z.-Z. Li, X. Xue, Y. Li, Y. Zhou, Y. Chen, C. Zhang, Y. Fan, Z. Wang, S. Huang, F. Piedrahita-Velez, Y. Liao, H. Wang, M. Yang, H. Ji, J. Wang, S. Yan, P. Torr, and L. Bai, "The landscape of agentic reinforcement learning for llms: A survey," *Transactions on Machine Learning Research*, 2026. [Online]. Available: https://mlanthology.org/tmlr/2026/zhang2026tmlr-landscape/

[2] A. Plaat, M. J. van Duijn, N. van Stein, M. Preuss, P. van der Putten, and K. J. Batenburg, "Agentic large language models, a survey," *Journal of Artificial Intelligence Research*, vol. 84, 2025. [Online]. Available: https://doi.org/10.1613/jair.1.18675

[3] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Transactions on Machine Learning Research*, 2024. [Online]. Available: https://mlanthology.org/tmlr/2024/wang2024tmlr-voyager/

[4] B. Bateni and J. Whitehead, "Language-driven play: Large language models as game-playing agents in slay the spire," in *Proceedings of the 19th International Conference on the Foundations of Digital Games (FDG 2024)*. New York, NY, USA: ACM, 2024.

[5] Z. Wang, S. Cai, G. Chen, A. Liu, X. Ma, and Y. Liang, "Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents," in *Advances in Neural Information Processing Systems*, 2023.

[6] Z. Wang, F. Wu, H. Wang, X. Tang, B. Li, Z. Yin, Y. Ma, Y. Li, W. Sun, X. Chen, and Y. Ye, "Why reasoning fails to plan: A planning-centric analysis of long-horizon decision making in llm agents," *arXiv preprint arXiv:2601.22311*, 2026. [Online]. Available: https://doi.org/10.48550/arXiv.2601.22311

[7] S. Mehri, X. Yang, T. Kim, G. Tur, S. Mehri, and D. Hakkani-Tür, "Goal alignment in llm-based user simulators for conversational ai," *Transactions of the Association for Computational Linguistics*, vol. 14, pp. 852–876, 2026. [Online]. Available: https://doi.org/10.1162/TACL.a.687

[8] S. Park and M. Kwon, "Multi^2: Hierarchical multi-agent decision-making with llm-based agents in interactive environments," *arXiv preprint arXiv:2606.03698*, 2026.

[9] WWA Wing, "Magic tower," https://web.archive.org/web/20171201030605/http://wwajp.com/mtower/, 2017, archived on Dec. 1, 2017. Accessed: Aug. 21, 2026.

[10] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, 2022. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html

[11] S. Ontañón, G. Synnaeve, A. Uriarte, F. Richoux, D. Churchill, and M. Preuss, "A survey of real-time strategy game ai research and competition in starcraft," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 5, no. 4, pp. 293–311, 2013. [Online]. Available: https://doi.org/10.1109/TCIAIG.2013.2286295

[12] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[13] K. Valmeekam, M. Marquez, and S. Kambhampati, "Can large language models really improve by self-critiquing their own plans?" *arXiv preprint arXiv:2310.08118*, 2023. [Online]. Available: https://arxiv.org/abs/2310.08118

[14] K. Stechly, K. Valmeekam, and S. Kambhampati, "On the self-verification limitations of large language models on reasoning and planning tasks," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=4O0v4s3IzY

[15] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.

[16] T. Wang, S. Gooding, F. Hartmann, O. Riva, and E. Grefenstette, "A subgoal-driven framework for improving long-horizon llm agents," *arXiv preprint arXiv:2603.19685*, 2026.

[17] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X

[18] R. Wang, P. Jansen, M.-A. Côté, and P. Ammanabrolu, "Scienceworld: Is your agent smarter than a 5th grader?" *arXiv preprint arXiv:2203.07540*, 2022.

[19] K. Valmeekam, M. Marquez, S. Sreedharan, and S. Kambhampati, "On the planning abilities of large language models: A critical investigation," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 75 993–76 005. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/efb2072a358cefb75886a315a6fcf880-Abstract-Conference.html

[20] Y. Liu, D. Li, K. Wang, Z. Xiong, F. Shi, J. Wang, B. Li, and B. Hang, "Are llms good at structured outputs? a benchmark for evaluating structured output capabilities in llms," *Information Processing & Management*, vol. 61, no. 5, p. 103809, 2024.

[21] C. Shorten, C. Pierse, T. B. Smith, E. Cardenas, A. Sharma, J. Trengrove, and B. van Luijt, "Structuredrag: Json response formatting with large language models," *arXiv preprint arXiv:2408.11061*, 2024.

[22] J. Yang, D. Jiang, L. He, S. Siu, Y. Zhang, D. Liao, Z. Li, H. Zeng, Y. Jia, H. Wang, B. Schneider, C. Ruan, W. Ma, Z. Lyu, Y. Wang, Y. Lu, Q. D. Do, Z. Jiang, P. Nie, and W. Chen, "Structeval: Benchmarking llms' capabilities to generate structural outputs," *arXiv preprint arXiv:2505.20139*, 2025.

[23] S. Geng, H. Cooper, M. Moskal, S. Jenkins, J. Berman, N. Ranchin, R. West, E. Horvitz, and H. Nori, "Generating structured outputs from language models: Benchmark and studies," *arXiv preprint arXiv:2501.10868*, 2025.

[24] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994.

[25] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972.

[26] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," *arXiv preprint arXiv:2508.10925*, 2025.

[27] Gemma Team, "Gemma 4 technical report," *arXiv preprint arXiv:2607.02770*, 2026.

[28] Qwen Team, "Qwen3.5: Towards native multimodal agents," Qwen Technical Blog, 2026. [Online]. Available: https://qwen.ai/blog?id=qwen3.5

[29] Kimi Team, "Kimi k2.5: Visual agentic intelligence," *arXiv preprint arXiv:2602.02276*, 2026.

[30] GLM-5 Team, "Glm-5: From vibe coding to agentic engineering," *arXiv preprint arXiv:2602.15763*, 2026.

[31] MiniMax, "The minimax-m2 series: Mini activations unleashing max real-world intelligence," *arXiv preprint arXiv:2605.26494*, 2026.

[32] DeepSeek, "Deepseek-v3.1 release," DeepSeek API Documentation, 2025. [Online]. Available: https://api-docs.deepseek.com/news/news250821

[33] Ollama, "Ollama cloud models," Ollama Documentation, 2025. [Online]. Available: https://ollama.com/blog/cloud-models

Appendices

A Mota1 Agent13 and Agent14 System Prompt

This appendix provides the complete system prompts for the agent versions discussed in Chapter 4 and Chapter 7. In the main text, only the prompt changes most relevant to the observed behavioral differences are quoted. The full prompts are included here for reproducibility and detailed comparison.

A.1 `mota1_agent13` System Prompt

Full system prompt for `mota1_agent13`

```
You are the decision core of a simplified Mota tower-game agent.
Your task is to choose exactly one next action.
Decision method:
- First follow the current stage.
- Then choose one target from candidate_targets when possible.
- Then choose one action from available_action_ids.
- Prefer stable progress over wandering.
- Avoid wasting steps in already-visited areas when a less-visited target
is available.
Output rules:
- Return JSON only.
- Do not output analysis, reflection, plans, risk notes, or explanations.
- Choose exactly one action from available_action_ids.
Output schema:
{ "stage": "collect_key", "target": "Yellow Key at (5,10)", "action":
"move_left" }
```

A.2 `mota1_agent14` System Prompt

Full system prompt for `mota1_agent14`

```
You are the decision core of a simplified Mota tower-game agent.
Your task is to choose exactly one next action.
Decision method:
- First follow the current stage.
- Then choose one target from candidate_targets when possible.
- Then choose one action from available_action_ids.
- If you choose a target that has a path_start, your action should usually
match that path_start.
- Do not say you are targeting stairs, keys, or doors and then move in a
different direction unless the path_start is unavailable.
- Prefer stable progress over wandering.
- Avoid wasting steps in already-visited areas when a less-visited target
is available.
```

```
- When a target is selected, prioritize consistency between target and
action over vague exploration.
Output rules:
- Return JSON only.
- Do not output analysis, reflection, plans, risk notes, or explanations.
- Choose exactly one action from available_action_ids.
Output schema:
{ "stage": "collect_key", "target": "Yellow Key at (5,10)", "action":
"move_left" }
```

B Full Logs and Traces: `mota1_agent13` vs. `mota1_agent14`

This appendix provides the complete run logs and the pivotal decision-trace excerpts referenced in the RQ7 case study (Chapter 7). Important raw trace files are included as supplementary digital files; this appendix reproduces the run logs in full and the Step 6 trace records that show the point of divergence between the two agents.

B.1 `mota1_agent13` Run Log

mota1_agent13_run_001.log

```
[agent_to_human] Run log file: logs\mota1_agent13_run_001.log
[agent_to_human] Trace file: logs\mota1_agent13_trace_001.json
New game started. Current position: M1 (6, 11), keys yellow 0.
Step 1: stage collect_key, choose to move up, target Yellow Key at (5,10).
Current state: M1 (6, 11), keys yellow 0.
Result: moved
State after action: position 6, 10 on M1, keys yellow 0.
Step 2: stage collect_key, choose to move left, target Yellow Key at (5,10)
    .
Current state: M1 (6, 10), keys yellow 0.
Result: got yellow key
State after action: position 5, 10 on M1, keys yellow 1.
Step 3: stage push_stairs, choose to move right, target Yellow Door at
    (6,9).
Current state: M1 (5, 10), keys yellow 1.
Result: moved
State after action: position 6, 10 on M1, keys yellow 1.
Step 4: stage push_stairs, choose to move up, target Yellow Door at (6,9).
Current state: M1 (6, 10), keys yellow 1.
Result: opened yellow door
State after action: position 6, 9 on M1, keys yellow 0.
Step 5: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (6, 9), keys yellow 0.
Result: moved
State after action: position 6, 8 on M1, keys yellow 0.
Step 6: stage push_stairs, choose to move left, target Stairs at (1,1).
Current state: M1 (6, 8), keys yellow 0.
Result: moved
```

```
State after action: position 5, 8 on M1, keys yellow 0.
Step 7: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (5, 8), keys yellow 0.
Result: moved
State after action: position 5, 7 on M1, keys yellow 0.
Step 8: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (5, 7), keys yellow 0.
Result: moved
State after action: position 5, 6 on M1, keys yellow 0.
Step 9: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (5, 6), keys yellow 0.
Result: moved
State after action: position 5, 5 on M1, keys yellow 0.
Step 10: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (5, 5), keys yellow 0.
Result: moved
State after action: position 5, 4 on M1, keys yellow 0.
Step 11: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (5, 4), keys yellow 0.
Result: moved
State after action: position 5, 3 on M1, keys yellow 0.
Step 12: stage push_stairs, choose to move down, target Stairs at (1,1).
Current state: M1 (5, 3), keys yellow 0.
Result: moved
State after action: position 5, 4 on M1, keys yellow 0.
Step 13: stage push_stairs, choose to move down, target Stairs at (1,1).
Current state: M1 (5, 4), keys yellow 0.
Result: moved
State after action: position 5, 5 on M1, keys yellow 0.
Step 14: stage push_stairs, choose to move down, target Stairs at (1,1).
Current state: M1 (5, 5), keys yellow 0.
Result: moved
State after action: position 5, 6 on M1, keys yellow 0.
Step 15: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (5, 6), keys yellow 0.
Result: moved
State after action: position 5, 5 on M1, keys yellow 0.
Step 16: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (5, 5), keys yellow 0.
Result: moved
State after action: position 5, 4 on M1, keys yellow 0.
Step 17: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (5, 4), keys yellow 0.
Result: moved
State after action: position 5, 3 on M1, keys yellow 0.
Step 18: stage push_stairs, choose to move down, target Stairs at (1,1).
Current state: M1 (5, 3), keys yellow 0.
Result: moved
State after action: position 5, 4 on M1, keys yellow 0.
Step 19: stage push_stairs, choose to move up, target Stairs at (1,1).
```

```
Current state: M1 (5, 4), keys yellow 0.
Result: moved
State after action: position 5, 3 on M1, keys yellow 0.
Step 20: stage push_stairs, choose to move down, target Stairs at (1,1).
Current state: M1 (5, 3), keys yellow 0.
Result: moved
State after action: position 5, 4 on M1, keys yellow 0.
Step 21: stage push_stairs, choose to move down, target Stairs at (1,1).
Current state: M1 (5, 4), keys yellow 0.
Result: moved
State after action: position 5, 5 on M1, keys yellow 0.
Step 22: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (5, 5), keys yellow 0.
Result: moved
State after action: position 5, 4 on M1, keys yellow 0.
Step 23: stage push_stairs, choose to move down, target Stairs at (1,1).
Current state: M1 (5, 4), keys yellow 0.
Result: moved
State after action: position 5, 5 on M1, keys yellow 0.
Step 24: stage push_stairs, choose to move down, target Stairs at (1,1).
Current state: M1 (5, 5), keys yellow 0.
Result: moved
State after action: position 5, 6 on M1, keys yellow 0.
Step 25: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (5, 6), keys yellow 0.
Result: moved
State after action: position 5, 5 on M1, keys yellow 0.
Step 26: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (5, 5), keys yellow 0.
Result: moved
State after action: position 5, 4 on M1, keys yellow 0.
Step 27: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (5, 4), keys yellow 0.
Result: moved
State after action: position 5, 3 on M1, keys yellow 0.
Step 28: stage push_stairs, choose to move down, target Stairs at (1,1).
Current state: M1 (5, 3), keys yellow 0.
Result: moved
State after action: position 5, 4 on M1, keys yellow 0.
Step 29: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (5, 4), keys yellow 0.
Result: moved
State after action: position 5, 3 on M1, keys yellow 0.
Step 30: stage push_stairs, choose to move down, target Stairs at (1,1).
Current state: M1 (5, 3), keys yellow 0.
Result: moved
State after action: position 5, 4 on M1, keys yellow 0.
Step 31: stage push_stairs, choose to move down, target Stairs at (1,1).
```

B.2 `mota1_agent14` Run Log

mota1_agent14_run_001.log

```
[agent_to_human] Run log file: logs\mota1_agent14_run_001.log
[agent_to_human] Trace file: logs\mota1_agent14_trace_001.json
New game started. Current position: M1 (6, 11), keys yellow 0.
Step 1: stage collect_key, choose to move up, target Yellow Key at (5,10).
Current state: M1 (6, 11), keys yellow 0.
Result: moved
State after action: position 6, 10 on M1, keys yellow 0.
Step 2: stage collect_key, choose to move left, target Yellow Key at (5,10)
    .
Current state: M1 (6, 10), keys yellow 0.
Result: got yellow key
State after action: position 5, 10 on M1, keys yellow 1.
Step 3: stage push_stairs, choose to move right, target Yellow Door at
    (6,9).
Current state: M1 (5, 10), keys yellow 1.
Result: moved
State after action: position 6, 10 on M1, keys yellow 1.
Step 4: stage push_stairs, choose to move up, target Yellow Door at (6,9).
Current state: M1 (6, 10), keys yellow 1.
Result: opened yellow door
State after action: position 6, 9 on M1, keys yellow 0.
Step 5: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (6, 9), keys yellow 0.
Result: moved
State after action: position 6, 8 on M1, keys yellow 0.
Step 6: stage push_stairs, choose to move right, target Stairs at (1,1).
Current state: M1 (6, 8), keys yellow 0.
Result: moved
State after action: position 7, 8 on M1, keys yellow 0.
Step 7: stage push_stairs, choose to move right, target Stairs at (1,1).
Current state: M1 (7, 8), keys yellow 0.
Result: moved
State after action: position 8, 8 on M1, keys yellow 0.
Step 8: stage push_stairs, choose to move right, target Stairs at (1,1).
Current state: M1 (8, 8), keys yellow 0.
Result: moved
State after action: position 9, 8 on M1, keys yellow 0.
Step 9: stage push_stairs, choose to move right, target Stairs at (1,1).
Current state: M1 (9, 8), keys yellow 0.
Result: moved
State after action: position 10, 8 on M1, keys yellow 0.
Step 10: stage push_stairs, choose to move right, target Stairs at (1,1).
Current state: M1 (10, 8), keys yellow 0.
Result: moved
State after action: position 11, 8 on M1, keys yellow 0.
Step 11: stage push_stairs, choose to move up, target Stairs at (1,1).
```

```
Current state: M1 (11, 8), keys yellow 0.
Result: moved
State after action: position 11, 7 on M1, keys yellow 0.
Step 12: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (11, 7), keys yellow 0.
Result: moved
State after action: position 11, 6 on M1, keys yellow 0.
Step 13: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (11, 6), keys yellow 0.
Result: moved
State after action: position 11, 5 on M1, keys yellow 0.
Step 14: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (11, 5), keys yellow 0.
Result: moved
State after action: position 11, 4 on M1, keys yellow 0.
Step 15: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (11, 4), keys yellow 0.
Result: moved
State after action: position 11, 3 on M1, keys yellow 0.
Step 16: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (11, 3), keys yellow 0.
Result: moved
State after action: position 11, 2 on M1, keys yellow 0.
Step 17: stage push_stairs, choose to move up, target Stairs at (1,1).
Current state: M1 (11, 2), keys yellow 0.
Result: moved
State after action: position 11, 1 on M1, keys yellow 0.
Step 18: stage push_stairs, choose to move left, target Stairs at (1,1).
Current state: M1 (11, 1), keys yellow 0.
Result: moved
State after action: position 10, 1 on M1, keys yellow 0.
Step 19: stage push_stairs, choose to move left, target Stairs at (1,1).
Current state: M1 (10, 1), keys yellow 0.
Result: moved
State after action: position 9, 1 on M1, keys yellow 0.
Step 20: stage push_stairs, choose to move left, target Stairs at (1,1).
Current state: M1 (9, 1), keys yellow 0.
Result: moved
State after action: position 8, 1 on M1, keys yellow 0.
Step 21: stage push_stairs, choose to move left, target Stairs at (1,1).
Current state: M1 (8, 1), keys yellow 0.
Result: moved
State after action: position 7, 1 on M1, keys yellow 0.
Step 22: stage push_stairs, choose to move left, target Stairs at (1,1).
Current state: M1 (7, 1), keys yellow 0.
Result: moved
State after action: position 6, 1 on M1, keys yellow 0.
Step 23: stage push_stairs, choose to move left, target Stairs at (1,1).
Current state: M1 (6, 1), keys yellow 0.
Result: moved
```

```
State after action: position 5, 1 on M1, keys yellow 0.
Step 24: stage push_stairs, choose to move left, target Stairs at (1,1).
Current state: M1 (5, 1), keys yellow 0.
Result: moved
State after action: position 4, 1 on M1, keys yellow 0.
Step 25: stage push_stairs, choose to move left, target Stairs at (1,1).
Current state: M1 (4, 1), keys yellow 0.
Result: moved
State after action: position 3, 1 on M1, keys yellow 0.
Step 26: stage push_stairs, choose to move left, target Stairs at (1,1).
Current state: M1 (3, 1), keys yellow 0.
Result: moved
State after action: position 2, 1 on M1, keys yellow 0.
Step 27: stage push_stairs, choose to move left, target Stairs at (1,1).
Current state: M1 (2, 1), keys yellow 0.
Result: reached stairs
State after action: position 1, 1 on M1, keys yellow 0.
Run finished: game ended. Total steps processed: 27.
```

B.3 Decision-Trace Excerpts at the Critical Fork (Step 6)

At Step 6 both agents are at position $(6, 8)$ with an identical `decision_context`, including `candidate_targets[0].path_start = "move_right"`. The excerpts below show the raw `llm_output.raw_payload.thinking` field recorded in each agent's trace file at this step.

Agent13 – Step 6 (`mota1_agent13_trace_002.json`)

```
"decision_context": {
  "stage": "push_stairs",
  "candidate_targets": [
    {
      "summary": "Stairs at (1,1)",
      "kind": "stairs",
      "steps": 22,
      "path_start": "move_right",
      "id": "upFloor"
    }
  ]
}

"llm_output.raw_payload.thinking":
"We have to output JSON with stage, target, action. The current
decision_context stage is 'push_stairs'. candidate_targets only one:
Stairs at (1,1). We are at position (6,8). Need to move towards that
target... From current (6,8), moving left seems good to go toward
column 1... Possibly left is best immediate... Thus choose action
'move_left'."

"decision": {
```

```
  "stage": "push_stairs",
  "target": "Stairs at (1,1)",
  "action": "move_left"
}
```

Agent14 – Step 6 (mota1_agent14_trace_001.json)

```
"decision_context": {
  "stage": "push_stairs",
  "candidate_targets": [
    {
      "summary": "Stairs at (1,1)",
      "kind": "stairs",
      "steps": 22,
      "path_start": "move_right",
      "id": "upFloor"
    }
  ]
}

"llm_output.raw_payload.thinking":
"We need to output JSON with stage, target, action. Current stage is
'push_stairs'. Candidate target is stairs at (1,1) with path_start
'move_right'. So ideally we should move right if available, matching
path_start. Available actions include move_right... Move right -> to
(7,8) which is '.' open. So move_right is valid. Thus output JSON."

"decision": {
  "stage": "push_stairs",
  "target": "Stairs at (1,1)",
  "action": "move_right"
}
```

C Prompt Templates

This appendix provides the main prompt templates used by the advanced Generation III agent configuration. The full prompts are reproduced here to support transparency and reproducibility.

C.1 Planning Prompt

PLAN_SYSTEM_PROMPT

```
You are the strategic planner for a Mota tower-climbing game agent.

Your job: analyze the current game state and decide the single best next
   target for the player to move to.
```

```
GAME RULES  (read carefully -- you must follow these to reason correctly)


1. GRID AND COORDINATES
   - Each floor is a 13x13 grid. Coordinates: (x, y) where x=column, y=row.
   - (0,0) is the top-left corner. x increases rightward, y increases
    downward.
   - Tile at column x, row y = floor_map[y][x]  (row first, then column).

2. MAP SYMBOLS
   #   WALL -- completely solid. Cannot be entered or crossed under ANY
               circumstances. No exceptions. No way through.
   .   Empty floor -- free to walk on.
   V   Player's current position.
   K   Yellow key  -- step on it to collect (+1 yellow key).
   b   Blue key    -- step on it to collect (+1 blue key).
   r   Red key     -- step on it to collect (+1 red key).
   D   Yellow door -- BLOCKED like a wall unless player has >=1 yellow key.
                      Stepping on it costs 1 yellow key and removes the
    door.
   B   Blue door   -- BLOCKED unless player has >=1 blue key.
   R   Red door    -- BLOCKED unless player has >=1 red key.
   U   Up-stairs   -- stepping on it INSTANTLY teleports player to the
                      connected tile on the floor above. Player never stays
     here.
   N   Down-stairs -- stepping on it instantly teleports to the floor below
    .

3. MOVEMENT RULES
   - The player moves ONE tile per step, in 4 directions only: up/down/left
    /right.
     No diagonal movement.
   - A step is LEGAL only if the destination tile is walkable:
       * NOT a wall (#)
       * NOT a locked door (door with no matching key in hand)
   - The player CANNOT jump over walls or doors. Every tile on the path
    counts.

4. HOW TO VERIFY A PATH IS VALID
   Before claiming "the player can reach X from Y", trace the route tile by
     tile:
     a. List each (x,y) tile you intend to step through.
     b. Look up that tile in the floor_map: floor_map[y][x]
     c. If any tile is '#' -> path is BLOCKED. Find a different route.
     d. If any tile is a door (D/B/R) -> check if player has that key.
        If not -> path is BLOCKED until key is collected.
   Only confirm a path after checking every single tile along it.
```

```
   WRONG reasoning example:
     "Go along row 5 from x=2 to x=8."
     -> If floor_map[5][4] = '#', the path is blocked at (4,5). You must
    detour.

   CORRECT reasoning example:
     "Check row y=5: (2,5)='.', (3,5)='.', (4,5)='#' <- WALL, blocked.
      Try alternative: go down to (2,6), right to (5,6), up to (5,5)...
      Check each: (2,6)='.', (3,6)='.', (4,6)='.', (5,6)='.', (5,5)='.' ->
    clear."

5. DOOR RULES
   - A locked door blocks movement exactly like a wall.
   - Once you have the matching key, stepping on the door removes it (costs
     1 key).
   - You cannot keep the key after opening a door -- it is consumed.
   - A door blocks from BOTH sides equally. There is no 'back side'
    approach
     that bypasses the key requirement.
   - Yellow door (D) requires yellow key (K).
   - Blue door (B) requires blue key (b).
   - Red door (R) requires red key (r).

6. STAIRCASE RULES
   - Stepping on U/N instantly teleports the player to another floor.
   - The player is NEVER standing on the staircase tile after stepping.
   - Therefore: NEVER set a staircase tile (U or N) as your target.
     The player teleports away and the target coordinate can never be
    reached.
   - To go to an upper floor: target an object ON the upper floor.
     The BFS pathfinder will automatically route through the staircase.

   Staircase connections:
     M1(1,1)U  <->  M2(1,1)
     M2(1,11)U <->  M3(1,11)
     M3(11,11)U <-> M4(1,11)
     M4(10,1)U  <->  M5(1,11)

7. KEY ECONOMY
   - Count how many keys of each color you need before opening required
    doors.
   - Collect reachable keys first if you are short.
   - Never assume you can open a door without confirming you hold the key.

8. EXPERIENCE MEMORY
   - historical_lessons contains soft-failure lessons inferred from
    previous
     runs. A soft failure means the run did not emit an explicit game-over,
     but still reached a resource-locked or progress-locked dead end.
```

```
   - Treat these lessons as high-priority warnings when they match the
    current
     map facts.
   - If a lesson says a key was wasted on a non-critical door and later a
     critical door of the same color became impossible to open, avoid
     repeating that pattern.


VICTORY CONDITION

  Reach M5 position (11,11)  [the final exit]


HOW TO REASON (step by step)

1. Find the player position (V) on the current floor map.
2. Trace backward from the victory tile: what doors block the path?
   What keys do those doors need? Where are those keys?
3. For each candidate target, check if it is likely reachable
   (no obvious wall barriers or missing keys blocking the path).
4. Pick the deepest reachable target on the critical path.
5. If you hold a key that can open a blocking door, that door is usually
   the top priority.
6. If the critical path is fully blocked, find the nearest reachable key
   that will eventually unblock something.

Return JSON only -- exactly this schema:
{
  "intent": "choose_target",
  "reasoning": "Tile-by-tile wall checks + subgoal chain explanation.
               Show which tiles you verified and why the path is clear.",
  "subgoal_chain": [
    "victory <- red door M4(11,3) <- red key M3(9,8) <- ...",
    "currently holding: yellow=1, so can open yellow door M1(6,9)"
  ],
  "target": {
    "floor": "M1",
    "x": 6,
    "y": 9,
    "type": "door",
    "color": "yellow"
  },
  "target_summary": "open yellow door at M1 (6,9)",
  "fallback_target": {
    "floor": "M1",
    "x": 5,
    "y": 10,
    "type": "key",
    "color": "yellow"
  },
```

```
  "reply_to_human": "one-sentence explanation for the human operator",
  "warning": ""
}
```

C.2 Reflection Prompt

REFLECT_SYSTEM_PROMPT

```
You are a reflection module for a game-playing agent.

You will receive structured evidence from a stalled or failed episode.
Infer the most likely decision error, explain the causal chain briefly,
and produce one compact lesson that may help future episodes.

Rules:
- Use only the evidence provided.
- Prefer general lessons over map-specific wording when possible.
- Do not assume hidden rules beyond the evidence.
- Output JSON only.

Schema:
{
  "primary_failure_type": "resource_budget_error|critical_path_error|
   reachability_error|loop_or_stall_error|target_semantics_error|other",
  "secondary_failure_types": [],
  "what_went_wrong": "short explanation",
  "causal_chain": ["step 1", "step 2"],
  "lesson": "short reusable lesson",
  "applicability": {
    "scope": "general|local",
    "confidence": 0.0
  },
  "evidence_used": ["fact 1", "fact 2"]
}
```

C.3 Human-Interaction Prompt

INTERACT_SYSTEM_PROMPT

```
You are the strategic planner for a Mota tower-climbing game agent.
The human operator has responded to your current target recommendation.

Your job: read the human message carefully and decide how to respond.

Intent types

- answer     : Human asked a question. Answer it, keep the current target.
- reconsider : Human pointed out an error or gave a better suggestion.
```

```
                You MUST change your target and explain why the human was
    right.
                Output the new target in new_target / new_target_summary /
                new_reasoning / new_subgoal_chain.
- execute    : Human accepted the recommendation (empty message or
    agreement).
                Proceed with the current target.
- quit       : Human wants to stop.

Important rules

- If the human says your reasoning is wrong, your coordinates are wrong, or
  proposes a different target -- use intent=reconsider and adopt the
    correction
  if it is consistent with the game facts provided.
- Do NOT keep defending the original target if the human provides a valid
  counter-argument backed by the map or key count.
- store_feedback=true when the human's message contains strategic
    preference
  worth remembering (e.g. "prefer red keys first", "avoid floor M3 for now
    ").

Return JSON only -- exactly this schema:
{
  "intent": "answer|reconsider|execute|quit",
  "reply_to_human": "concise reply to the human",
  "store_feedback": false,
  "feedback_summary": "short summary if store_feedback is true, else empty
    ",
  "new_target": {"floor": "M1", "x": 6, "y": 9, "type": "door", "color": "
    yellow"},
  "new_target_summary": "open yellow door at M1 (6,9)",
  "new_reasoning": "explanation of why the new target is better",
  "new_subgoal_chain": []
}
```